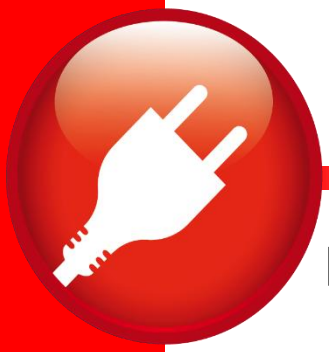
DIPLOMA THESIS



8Bit-Computer

Higher Department of Electrical Engineering

Executed by:

Florian Silberbauer, 5AHET

Simon Schmidtgrabner, 5AHET

Supervised by:

DI Bernhard Breinbauer

Practical Supervisor:

Ing. Reinhard Pölzleithner

Submission: 27. March 2026

8Bit-Computer

2025/26

bildung mit zukunft

HÖHERE TECHNISCHE BUNDESLEHRANSTALT WELS

WWW.HTL-WELS.AT

Diploma students

Name: Florian Silberbauer

Date of Birth: 18.01.2007

Address: Heiderosenstraße 7, 4614 Marchtrenk

E-Mail: silberbauerflorian366@gmail.com

Name: Simon Schmidtgrabner

Date of Birth: 27.07.2007

Address: Albrechtstraße 29, 4614 Marchtrenk

E-Mail: schmidtgrabner.simon@gmail.com

Supervisors

Name: DI Bernhard Breinbauer

E-Mail: bernhard.breinbauer@bildung.gv.at

Name: Ing. Reinhard Pölzleithner, BEd

E-Mail: reinhard.poelzleithner1@bildung.gv.at

Acknowledgement

To express our deep appreciation for all of those who assisted us with the completion of this diploma thesis, thank you to our supervisors, DI Bernhard Breinbauer and Ing. Reinhard Pölzleithner, BEd., for their regular assistance, constructive critique and continuous support throughout the entire process. We would like to express gratitude to our school HTLBA Wels, for providing the opportunity and the resources needed for conducting this research. Lastly, we would like to thank our family and friends, who were there throughout this process, providing us with their continued support, understanding and encouragement, allowing us to stay focused and to be able to complete this project. Otherwise, we would not have accomplished this outcome of this diploma thesis.

Eidesstattliche Erklärung

Hiermit versichern wir, die vorliegende Arbeit selbständig, ohne fremde Hilfe und ohne Benutzung anderer als der von uns angegebenen Quellen angefertigt zu haben. Alle Stellen, die wörtlich oder sinngemäß aus fremden Quellen direkt oder indirekt übernommen wurden, sind als solche gekennzeichnet.

Wels am _____Name _____

Unterschrift _____

Wels am _____Name _____

Unterschrift _____

Kurzfassung

Diese Arbeit befasst sich mit dem Entwurf, der Simulation und der Hardwareimplementierung eines 8-Bit-Computers mit einem dedizierten Befehlssatz. Das Hauptziel war die Entwicklung eines voll funktionsfähigen, vom Benutzer programmierbaren Computers, der aus grundlegenden digitalen Logikgattern aufgebaut ist. Das Projekt beginnt mit einem Überblick über wesentliche Konzepte der digitalen Elektronik, einschließlich Logikgattern, Flipflops, Binärzahlen und Zeitverhalten.

Der Kern der Arbeit konzentriert sich auf das logische Design des Computers, das aus einer zentralen Verarbeitungseinheit (CPU), einem Festwertspeicher (ROM), einem Arbeitsspeicher (RAM), Registern, Bussen und einer arithmetisch-logischen Einheit (ALU) besteht. Ein wichtiges Designmerkmal ist die Ausführung jedes Befehls innerhalb eines einzigen Taktzyklus. Das System wurde zunächst mit Simulationssoftware entwickelt und verifiziert, was iterative Tests einzelner Schaltungen ermöglichte.

Aufbauend auf dem simulierten Design geht das Projekt zur Hardware-Realisierung über, wobei integrierte Schaltungen aus der Logikfamilie der 7400er-Serie verwendet werden. Dies umfasst die schematische Konstruktion, die Entwicklung der Leiterplatte (PCB), die Montage und die Implementierung zusätzlicher Schaltungen wie Taktgeneratoren und einer Stromversorgung. Zusätzlich wurde ein benutzerdefinierter Assembler entwickelt, um die Verwendung für Menschen lesbarer Anweisungen zur Programmierung des Computers zu ermöglichen.

Das fertige System demonstriert die erfolgreiche Integration theoretischer Prinzipien und praktischer Ingenieurskunst, was zu einem funktionierenden 8-Bit-Computer führt, der benutzerdefinierte Programme ausführen kann. Dieses Projekt verdeutlicht sowohl die Komplexität als auch den Bildungswert des Baus eines Computers aus grundlegenden Komponenten und gibt Einblicke in Prozesse der unteren Ebene der Datenverarbeitung und das Design digitaler Systeme.

Abstract

This thesis involves the design, simulation, and hardware implementation of an 8-bit computer with a dedicated instruction set. The primary objective was to develop a fully functional, user-programmable computer constructed from fundamental digital logic gates. The project begins with an overview of essential concepts in digital electronics, including logic gates, flip-flops, binary numbers, and timing behaviour.

The core of the work focuses on the logical design of the computer, made up of a central processing unit (CPU), read-only memory (ROM), random access memory (RAM), registers, buses, and an arithmetic logic unit (ALU). A major design feature is the execution of each instruction within a single clock cycle. The system was first developed and verified using simulation software, enabling iterative testing of individual circuits.

Building upon the simulated design, the project proceeds to hardware realization using integrated circuits from the 7400-series logic family. This includes the schematic design, printed circuit board (PCB) development, assembly, and implementation of additional circuitry such as clock generators and a power supply. Additionally, a custom assembler was developed to enable human-readable instructions to be used for programming the computer.

The completed system demonstrates the successful integration of theoretical principles and practical engineering, resulting in a working 8-bit computer capable of executing user-defined programs. This project highlights both the complexity and educational value of constructing a computer from basic components, providing insight into low-level computing processes and digital system design.

Table of Contents

1 Introduction

1.1 Initial Situation

As part of the diploma thesis, a demonstrator for CPU architectures is to be developed. This will include an 8-bit computer made up of logic chips which can be used as a teaching aid in FI classes.

1.2 Objective of the Diploma Thesis

The aim of this thesis is to gain new insights in the field of computer technology through the design process. The structure and functionality of individual computer components, as well as the feasibility of a self-developed instruction set, will be examined. In addition, a custom arithmetic logic unit will be designed to execute commands such as addition, subtraction, comparison, and various logic operations. A decentralised control unit will also be developed.

1.2.1 Composition of the Work (Florian Silberbauer)

Planning and Simulation of logical circuits, Design of a command set, Writing a Test-Programme, Demonstration Programmes

1.2.2 Composition of the Work (Simon Schmidtgrabner)

Market research, Circuit Design, design of the PCB layout, Design of the programming device, Planning and building the demonstration device

2 Ideation

As it is evident that the school currently does not have any demonstrators for Microcontrollers or CPUs, it seems important for subsequent generations to have access to such. So, the idea behind this project is to create an as easily understandable computer as possible, so that it could be fully understood and programmed by external individuals. With the completed project, both the hardware and software will be provided to the school to enable the subsequent HTL students to have practical access to this topic and to deepen their understanding of it.

3 Logical Design – Florian Silberbauer

The subject of this thesis is a custom-built 8-bit computer with its own instruction set, along with a special architecture to make use of it. Every single one of its commands is meant to be executed in a single tick. This computer, like many of the sort, is made up of permanent storage, random access storage, a central processing unit and some input and output devices.

3.1 Basics

This chapter is meant to help further the understanding of the basic knowledge required for building a computer. These chapters need to be understood by heart in order to have an easier time understanding the project at hand.

3.1.1 Logical States

This computer, as most are, is reliant on binary or logical high and logical low signals, also often called ‘high’ and ‘low’ or ‘1’ and ‘0’. They are usually defined as voltages, with high being a voltage above a certain threshold and low being below a certain threshold.

3.1.2 Truth Tables

Truth tables are used in order to explain the function of simple logical circuits by showing a correlation between inputs and outputs. Usually, all the inputs are located on the left side, and the outputs are located on the right.

Here the input is written in the A column, and the output is written in the Q column. This truth table is meant to show the function of a NOT gate, which inverts the logical signal found at its input.

A	Q
0	1
1	0

Table 1: Truth Table of a Not Gate

3.1.3 Timing and Timing Diagrams

Proper timing is needed so that all the parts of the computer are able to finish their tasks and to avoid any unpredictable behaviour. Timing diagrams are meant to make the timing more understandable by showing the changes in multiple digital signals over time. There are multiple patterns meant to show different states of a signal.

The timing diagrams here were designed using an Open-Source software called Wave Drom.

3.1.3.1 Clock Signal

Figure 1: A Clock Signal

A clock is used in many different digital circuits in order to, as mentioned above, avoid unpredictable behaviour. It is a signal which oscillates between low and high at a defined frequency. It usually has a period of 50 percent, so it is off for the same amount of time as it is on. This signal is required by certain parts of the computer so that they can synchronize to the rest of the computer.

3.1.3.2 Undefined

This is usually meant to show that the signal is in an unknown state.

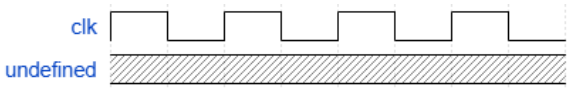


Figure 2: Undefined Signal

3.1.3.3 Transitions

In order to show that a value needs time to change to a new state, these transitions are used. They are the up-and-down slopes shown in the signal here and are usually heavily exaggerated.

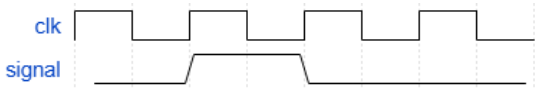


Figure 3: Signal Transition

3.1.3.4 Data

This depiction shows a binary value on a bus. The actual state all the bits are in is not of importance, only the timing of the value matters. The value is marked with a name in the data block. There are also transitions in this kind of signal.

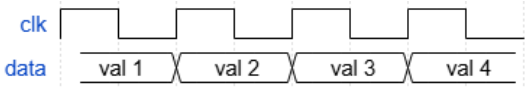


Figure 4: Data Signal

3.1.3.5 Falling and Rising Edges

These are the transitions between low and high states, which some circuits detect to improve the timing accuracy. A falling edge is defined as the transition of a high state to a low state. It is "falling" from the higher potential to the lower one. A rising edge is the other way around, so it rises from the lower potential to the higher one.

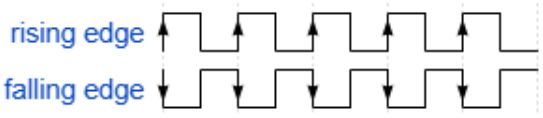


Figure 5: Rising and Falling Edges

3.1.4 Logic Gates

They are the building blocks used within all complex logical circuits for their simplicity and ease of use. The basic versions of these logical devices usually have two inputs and one output, with one exception being the NOT gate, having only one input and one output.

3.1.4.1 Not Gates

A	Q
0	1
1	0

Table 2: Truth Table of a Not Gate

The output on the right is always the opposite of the input on the left, so these kinds of logic gates are used for inverting signals.

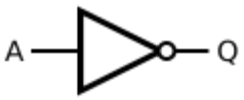


Figure 6: The Symbol of a Not Gate

(InductiveLoad, 2009)

3.1.4.2 And Gates

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

Table 3: Truth Table of an And Gate

Here, the output Q will only be high if the two inputs, A and B, are both high. This behaviour is sometimes used to only allow a signal to pass if a certain condition is met.

Figure 7: The Symbol of an And Gate

(InductiveLoad, 2009)

3.1.4.3 Or Gates

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

Table 4: Truth Table of an Or Gate

This gate outputs a high state if any of its inputs are high. Functionally, it is not like the word 'or', as it is used in conversation, with which it can be explained as either A or B, or both A and B having to be high for Q to be active.

Figure 8: The Symbol of an Or Gate

(InductiveLoad, 2009)

3.1.4.4 Exclusive Or Gates

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Table 5: Truth Table of an Xor Gate

An XOR gate's output is high if only one of the inputs is high. It is a little more like the word 'or' in common use.

Figure 9: The Symbol of an Xor Gate

(InductiveLoad, 2009)

3.1.5 Flip-flops

Sometimes also called "latches", these simple devices are used to store single-bit values. Since storing happens over extended periods of time, they are hard to depict using truth tables. Instead, timing diagrams are used.

There are more latches than are listed here; these are just the ones used within the design.

3.1.5.1 D-Flip Flop

A D-type flip-flop is usually used in order to store single-bit values with a clock signal to control the timing. The D input is meant for the signal to be stored, while the C input marked with the triangle is the clock input. The two outputs are on the right. The top one is the normal Q output, which shows the value stored, while the lower one is the inverted version (see 3.1.4.1 Not Gates) of that output.

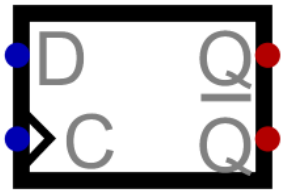


Figure 10: The Symbol of a D-Latch

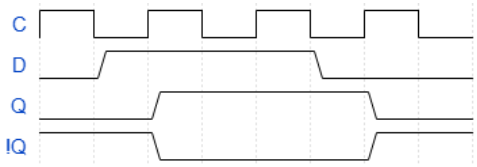


Figure 11: Timing Diagram of a D-Latch

Here, the stored value of the flip-flop is first assumed to be zero. As can be seen in the diagram, with a constant clock signal in sync with the data input, said input is delayed by half a cycle of the clock signal. The inverted Q output is marked with an exclamation mark, as is common in many programming languages such as C or Java.

3.1.5.2 SR-Latch

The S and R in the name stand for 'set' and 'reset', which are the functions of the two inputs. While the set input is high, the stored value will be set to high; if it already is high, nothing will change. If there is a high logical level on reset, the internal value is reset to zero. This device also has a normal Q output showing the actual stored value and an inverted version showing the opposite value.

Figure 12: The Symbol of a D-Latch

There are also SR latches with clock inputs. These, however, were not used in the logical design of the computer.

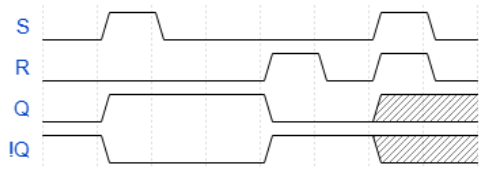


Figure 13: Timing Diagram of an SR-Latch

At the beginning, the stored value is assumed to be zero. Since this depiction has no clock input, the changes to the stored values are instant. If there is no dominant input, a problem arises when both inputs are high at the same time.

3.1.6 Binary and Hexadecimal

When working on anything related to computer science, a solid understanding of binary and hexadecimal numbers is important.

3.1.6.1 Decimal

Decimal numbers of a certain size are made up of multiple digits, and since there are ten different number symbols, each digit has a power of ten as a value.

For example, the number 142 is made up of the following values:

Places		Digits	Value
1	*	2	= 2
10	*	4	= 40
100	*	1	= 100
Sum			142

Table 6: The Decimal Digits of the Number 142

3.1.6.2 Binary

Binary numbers are similar to decimal numbers, with the major difference being the fact that every place within the number can only be a zero or a one. Because of that, the value of every place changes to powers of two. In that fashion, the maximum value achievable with 8 bits is 255.

Places		Digits	Value
1	*	0	= 0
2	*	1	= 2
4	*	1	= 4
8	*	1	= 8
16	*	0	= 0
32	*	0	= 0
64	*	0	= 0
128	*	1	= 128
Sum			142

Table 7: The Binary Digits of the Number 142

As can be seen from this table, the number 142 is written as **10001110** in binary.

3.1.6.3 Hexadecimal

In hexadecimal, there are 16 symbols, ranging from 0 to 9 and then from A to F. Because of that, the values of the places now scale to powers of 16. The reason for its common usage lies in the fact that a single hexadecimal value represents exactly 4 bits, which is of great help for working with binary numbers.

Places		Digits	Value
1	*	E	= 14
16	*	8	= 128
Sum			142

Table 8: The Hexadecimal Digits of the Number 142

The number 142 is written as E8 in hexadecimal. Sometimes numbers of this format are marked by adding '0x' in front of the number.

A common method for turning decimal numbers into hexadecimal ones is to write these numbers in binary and then look at these values in groups of 4 bits. It is then possible to replace these values with the corresponding hexadecimal symbols.

Number	142							
Binary	1	0	0	0	1	1	1	0
Decimal	14				8			
Hexadecimal	E				8			

Table 9: Translating between Decimal and Hexadecimal Numbers

3.2 Structure of the Computer

Most of the work went towards a custom CPU to be used in the computer. It is made up of volatile storage, or RAM, which can be accessed by the programme in order to store values; permanent storage for storing programmes; and an arithmetic logic unit, or ALU, which executes all the arithmetic or logical operations required by the code, such as adding, subtracting, incrementing or any of the logical operations explained in chapter 3.1.4 (Logic Gates). For user interaction, there are 8 light emitting diodes or Leds used to output values in a binary format (see chapter 3.1.6 Binary and Hexadecimal). For inputting values, there are 8 switches, by which binary values can be entered.

3.2.1 Buses

Buses are data wires, which have more than two devices connected, with the intention of communication between devices. In this computer, parallel buses are used, meaning every bus is made up of multiple wires, carrying digital data signals. There is a total of three buses used in this computer: the command bus, the data bus, and the address bus. All of these buses are connected to the ROM storage. Their width (the number of wires next to each other) is either 8, for the command and data buses, or 16 for the address bus.

3.2.1.1 Command Bus

This bus has a width of 8 bits and carries the command to be executed at the moment. It connects a part of the ROM storage to the multitude of circuits spread around the computer.

3.2.1.2 Data Bus

The data bus carries data for different commands to work with. If a command requires to return a value after an operation, for example after an addition, the ROM storage will be disconnected from the bus in order to avoid multiple different values being written to it, leading to confusion.

3.2.1.3 Address Bus

This bus is rarely used but serves to make certain commands simpler in execution. Most of the commands which are saving values to storage need data and address lines in order to find the relevant position in storage for the data on the bus to be written to.

3.2.2 ROM

The Read Only Memory (or ROM) is where the programme is stored. It consists of two 16-bit Rom chips, which can only be read from. At their input, there is the line number, coming from the programme counter. This number is incremented every cycle and can be thought of as the current line of the programme being executed. One of these chips is responsible for writing to the address bus and the other one is used for the data and command busses (see 3.2.1 Buses).

When a command is required to write to the data or address bus, the correlating part of the memory will be disabled to avoid multiple values on the bus at the same time.

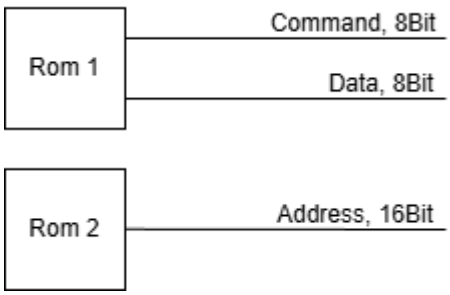


Figure 14: Rom Connections to the Busses

3.2.3 Command Circuits

Every command is implemented as a digital circuit, which has a comparator comparing the Opcode (unique value of a command) to the value on the command bus. These circuits are all connected to the command bus along with the data, or address bus depending on which operation the command is meant to execute.

3.2.4 Registers

A register is a way to store data. Unlike the ROM module, they usually only have one cell to store data to, which removes the need for an address to load or store values. All the registers in this computer are 8 bits wide. There are 9 registers within the computer, each having a specific purpose.

3.2.5 Arithmetic Logic Unit

The ALU, also called the Arithmetic Logic Unit, performs all the logical operations required for the computers' operation. It is made up of three input registers also called the A, B and C registers, to which all the internal commands are connected in some way. These internal commands usually manipulate data in some way, which they can permanently access from the input registers. When they are called, they take the data from the registers given to them, perform their operations, and output the result to the data bus.

3.2.6 A Single Computation Cycle

A computation cycle begins with the rising edge of the clock signal and ends with the next rising edge. Every computation cycle the values on the 3 buses will first synchronously change to the next values (see 3.2.2 ROM). Then, every command circuit compares their Opcode to the value on the command bus. Next, the command with the corresponding value starts operating. If it requires data, it will, synchronously with the clock signal, read data from the data bus. If it requires an address, it will read from the address bus as well. The command requires some time to finish executing. Afterwards it will finish executing at the next rising edge of the clock. If this command needs to return a value however, for example after it has finished an arithmetic operation, it will start outputting the value to the data or the address bus on the falling edge of the clock, until the next falling edge of the next command, so it will continue right into the next computation cycle.

While it is writing the value to the bus, it will send a signal to the ROM module to stop outputting values, as it would otherwise corrupt the values on the bus.

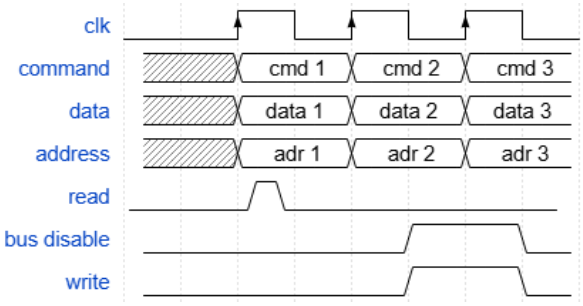


Figure 15: Computer Timing Diagram

3.3 Simulating the Computer

Since computers are complicated, the ability to design circuits and test them while doing so is nearly a requirement for this project. For this purpose, a fitting software was needed.

3.3.1 Software

The software that was chosen was an open-source project called “Digital” on GitHub. It was chosen for its ease of use and the ability to perform real time simulation on the design.

3.3.2 Method of Operation

Once opened, the software shows a circuit editor where the user can place components and connect them using wires. Once the circuit is complete, the user can start simulating it. During the simulation, the inputs can be manipulated in real time. A finished circuit can also be imported into other ones, with the in- and outputs used in the simulation also being used for interaction in the new circuits.

3.3.3 Common Symbols within the Software

For further use, the symbols present within the simulation need to be explained.

3.3.3.1 In- and Outputs

These symbols are used within the software to test a circuits’ behaviour. They can be named to help with understanding a circuit. When imported, they become the in- and outputs of the imported circuit.

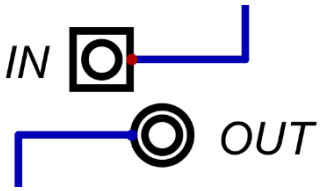


Figure 16: In- and Output Components in the Simulation

3.3.3.2 Wires and Splitters

Wires can be single or parallel. By using splitters, a parallel wire can be split into single wires, and multiple single wires can be combined into one parallel wire. However, there is no way in the circuit editor to see if a wire is parallel or not by only looking at the wire. Parallel wires are used heavily in this project, as are single wires.

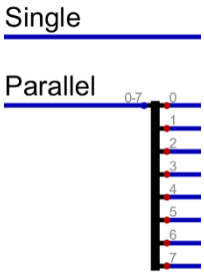


Figure 17: Wires and Splitters

3.3.3.3 *Drivers*

These devices are used to enable or disable signals. The ones used in this project are primarily parallel ones. The line at the side of the arrow denotes the enable input, while the input is at the left side of the arrow, and the output is on the other side.

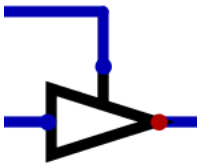


Figure 18: Symbol for a Driver

In	En	Out
Val	0	Z
Val	1	Val

Table 10: Truth Table for a Driver

When the enable signal is high, it will allow the value on the input through to the output, when it is low it will go into high impedance, so it is electrically as if it were disconnected.

3.3.3.4 *Multiplexers*

Multiplexers are used in order to select one signal from multiple inputs. The inputs at the left are the values available for selection, and the input at the bottom is used for choosing the output. The input at the bottom needs to have enough bits in order to give every input its unique value. Here, since there are only two inputs, the selection input just needs to have a width of one.

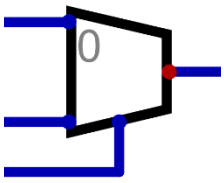


Figure 19: Symbol for a Multiplexer

3.3.3.5 *Custom Components*

It is possible to create custom components by importing existing circuits. This helps with compartmentalizing a big project such as this one, making it easier to find problems or errors within the design. Any changes made to the circuit imported as a custom component will directly be reflected in the custom component.

Figure 20: A Custom Compone

3.4 Circuits

3.4.1 Important Circuits

The commonly used circuits within the design are explained in this chapter.

3.4.1.1 Command Decoder

Every command has its own command decoder, which activates the command when needed. The circuit is made up of a comparator, with two 8-bit inputs and a single binary output. One of these inputs is set to a permanent value, while the other one is connected to the command bus. The output of the circuit is connected to various devices within the command circuit, in order to activate certain functions. If the permanent value is now detected on the bus, the command becomes active.

This example is from Digital. The block shown here is a comparator with three different outputs. Some of these are not used. It can be seen that the b-input is set to a permanent 8-bit value, while the a-input is connected to the command bus. The equal output is used to activate the command.

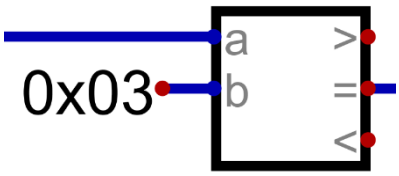


Figure 21: The Command Decoder

3.4.1.2 Output Circuit

Since commands which output to either the data or address bus need to begin at the falling edge of the current command and end at the falling edge of the next command, it is possible to delay the command decoder output for half a cycle. This delayed signal shows the exact time during which the respective bus needs to be disabled and commands need to output their data.

Since the output continues into the next cycle, the inputs of the command circuit can change while outputting. This would destroy the data being output, so the calculated data needs to be buffered in order to avoid this.

A buffer is temporary storage which is meant to hold data for a short while, allowing other circuits to work with it.

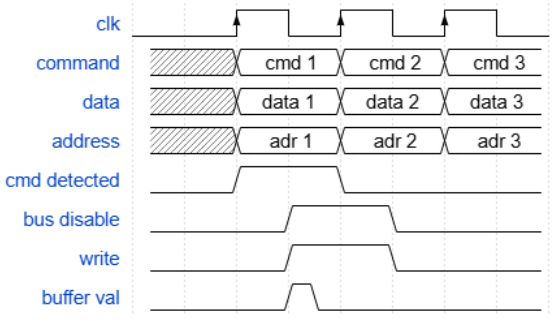


Figure 22: Output Timing

Here, in this depiction of a parallel not command, the A bus is inverted and written to the register, where it is buffered for half a cycle. This is achieved by inverting the clock signal. It now has a rising edge instead of a falling one. The enable signal, here called “EN”, which is supplied by the command decoder (see [3.4.1.1 Command Decoder](#)), will now be stored on the new rising edge, which is half a cycle later than before. The moment the value is stored in the D-Latch, it is transferred to the register and driver combination, which stores the value and writes it to the output. On the rising edge of the inverted clock signal, the new state of the enable signal will be stored, which is low, since there is no reason to execute the same command twice in a row. Once the low value is stored, the driver is disabled, and data stops being written to the output.

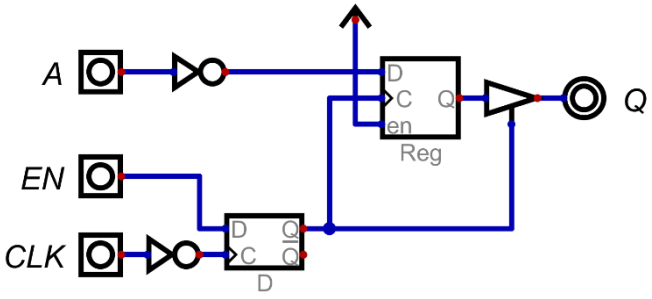


Figure 23: An Output Circuit

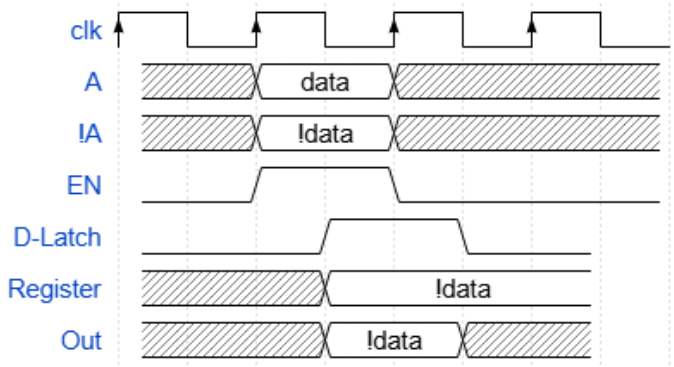


Figure 24: Timing Diagram for Output Circuits

3.4.2 Computer Overview

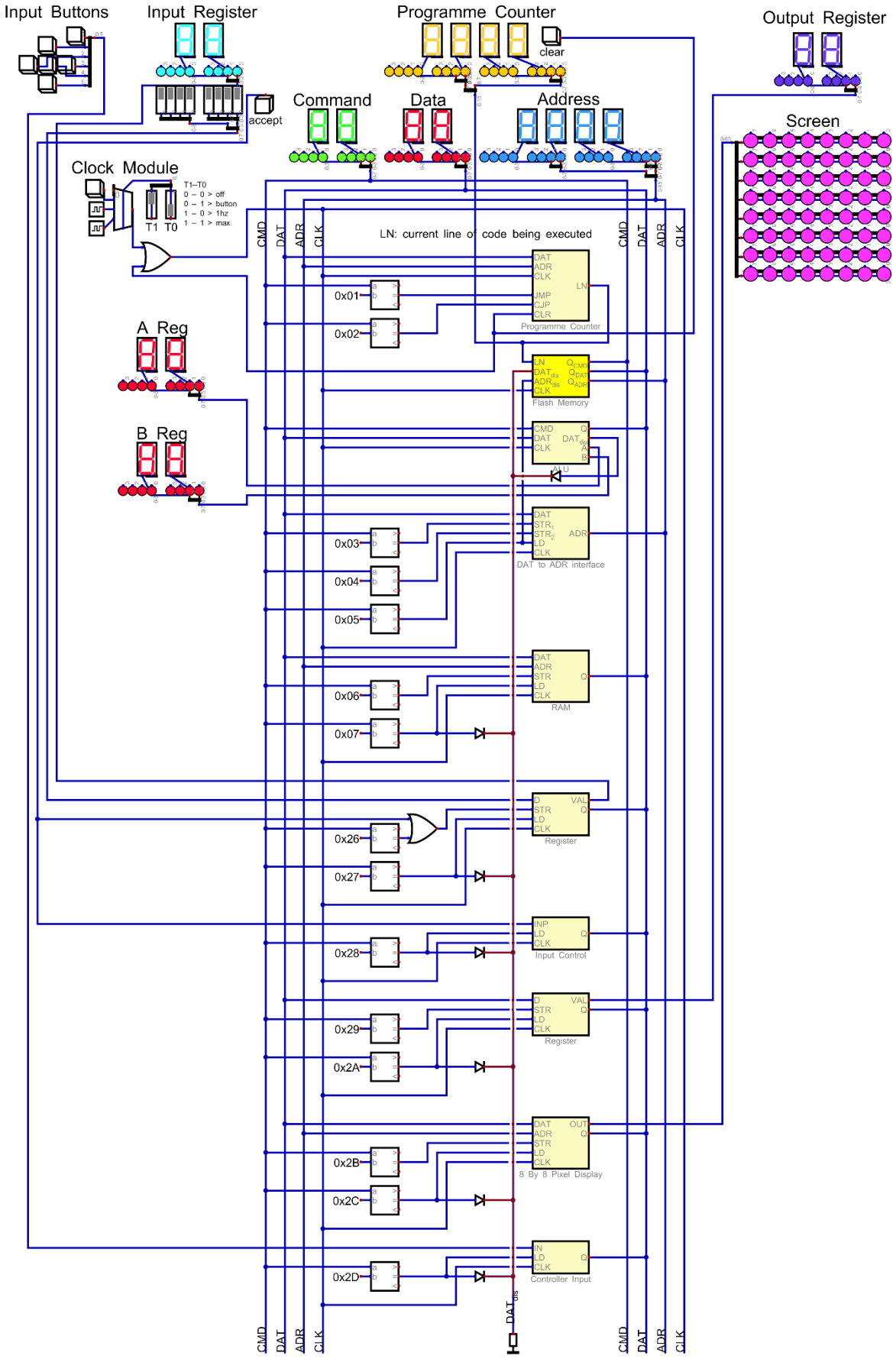


Figure 25: Computer Overview

This circuit has all the commands, which will soon be explained, implemented. Most of it is related to the CPU, such as the ALU, the Programme Counter, and the Data to Address Interface. The data disable line (here marked in red) is connected to all the commands writing to the data bus, as well as to the ALU, since most commands writing to the data bus are located there.

The yellow custom component is the ROM, where the programme is stored. It is marked in yellow, since the path of the binary files, with which the computer is programmed can be defined. This is useful in case the user needs to load a different programme.

The Input Buttons and the Screen depicted here were not implemented in the physical version of this computer. They are just meant to serve as an example for possible extensions to it.

3.4.3 Storage

3.4.3.1 ROM

This circuit implements the function explained in chapter 3.2.2 ROM.

The LN input is 16 bits wide and comes from the programme counter. The data and address disable lines are from the commands that need to disable these buses (as explained in 3.2.2).

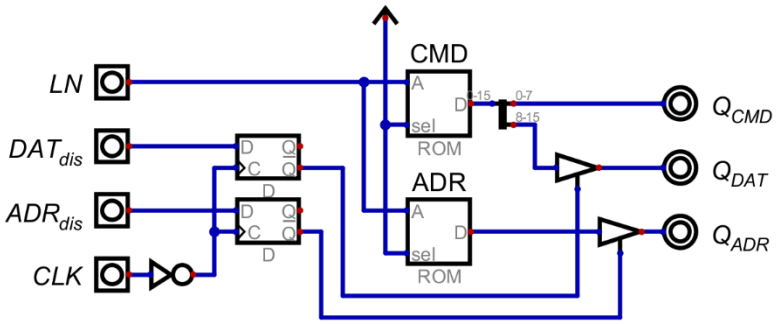


Figure 26: The ROM Circuit

If any commands send the signal for disabling a bus, it will be stored on the falling edge of the clock (as in chapter 3.4.1.2 Output Circuit) and stored for a full cycle until it is overwritten with another value on the next falling edge of the clock. While a disable signal is stored, the respective driver to the bus is disabled. This is not possible for the command bus, as commands need to be accessible all the time.

3.4.3.2 RAM

This module is meant to make the Ram component included in the software behave in a way that is easier to work with.

The D pin on the Ram component is a bidirectional data in- and output. The driver at the top of the circuit is meant to enable data input only

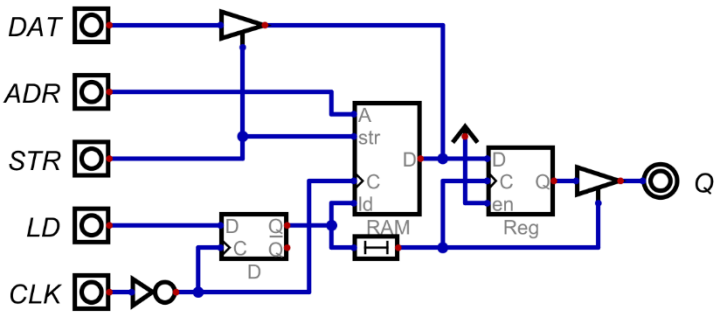


Figure 27: The Ram Circuit

when it is meant to be stored. The register along with the driver at the output are part of the output circuit (see chapter 3.4.1.2 Output Circuit). The delay between the D-Latch and the output register exists, so that the value from Ram can be fully loaded before being stored, avoiding undefined values. When loading values this module needs the relevant address to only be on the bus until the command is received.

3.4.4 Control Logic

The circuits explained here are used for controlling the flow of programmes or the flow of data between busses.

3.4.4.1 Programme Counter

The main part of the programme counter is the counter, which has its output connected to the address input to the ROM module (see 3.4.3.1 ROM).

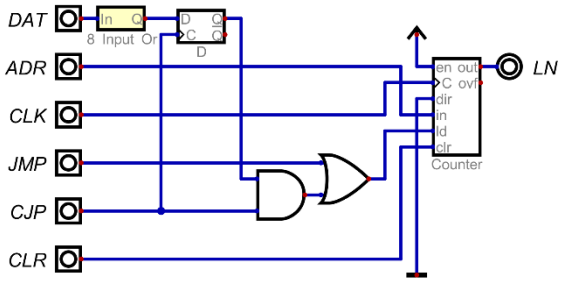


Figure 28: Programme Counter Circuit

Important commands in computers are jump commands. In this one, there are the normal jump command, which needs an address, to which it will jump, which means the value on the address bus will be loaded into the programme counter. Another one is a conditional jump, which only jumps to the specified address if the output of the 8 input or gate connected to the data bus is high.

3.4.4.2 Data to Address Interface

This circuit is meant to store data from the data bus and load it to the address bus, mainly to transfer data, which can be altered by different commands in order to implement behaviour for things like arrays or lists.

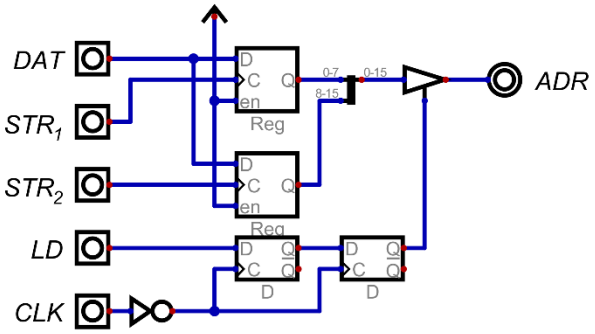


Figure 29: Circuit for Data to Address Register

In order to achieve that, two registers are put next to each other, and connected to the address bus together, in order to have access to all 16 bits of the address space. The second D-latch is there, in order to delay the output to the address bus, for other commands to be able to load values to the data bus. Otherwise, there would only be an address available with no data to store anywhere. This problem was first overlooked, since there was no proper programme making use of this command, so it was discovered quite late.

3.4.5 Arithmetic Logic Unit

3.4.5.1 Overview

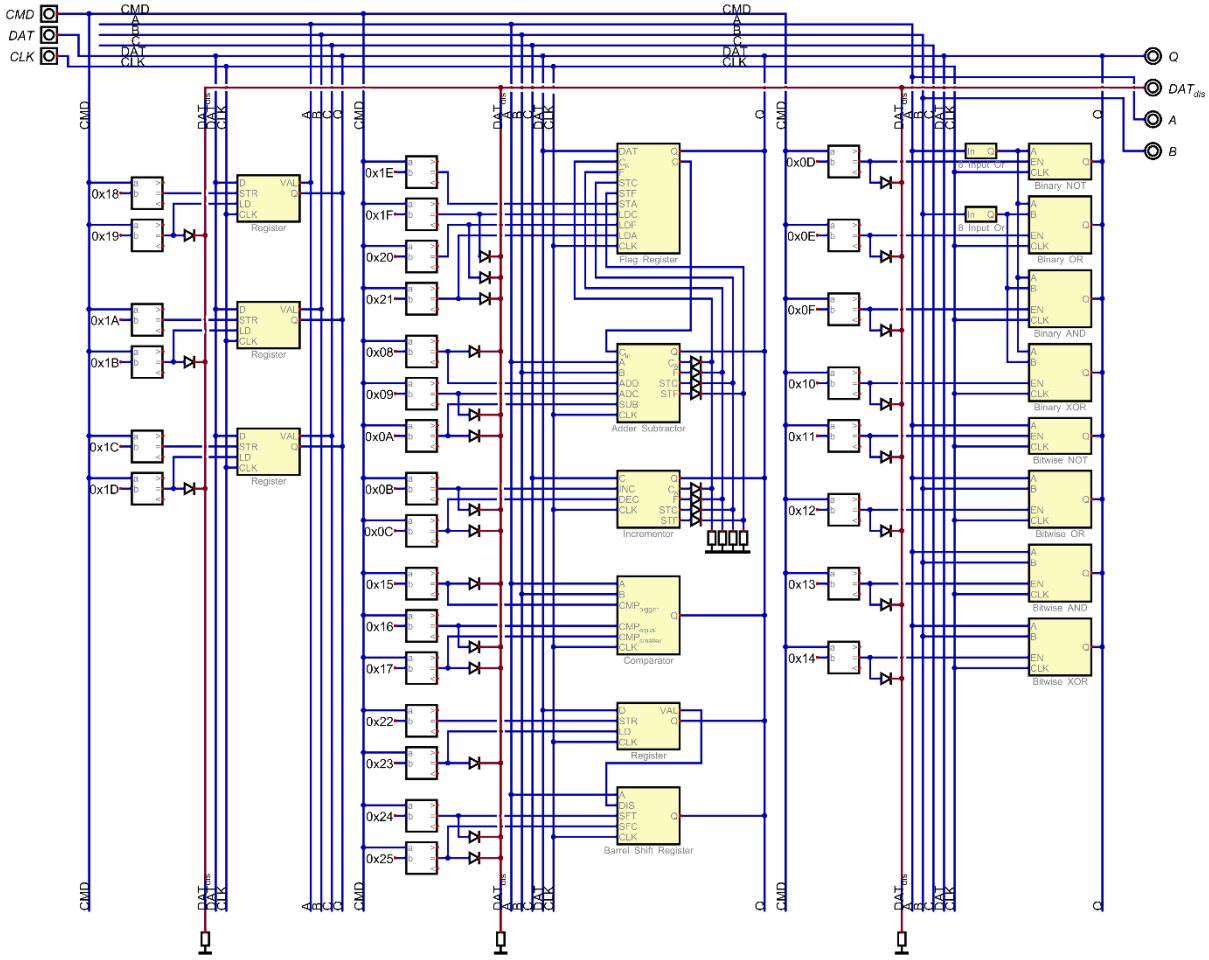


Figure 30: Alu Overview

In this file, all the commands which write to the data bus are connected via diodes to the Data Disable line, here marked in red, which is pulled low using pulldown resistors. These pulldown resistors give the line a defined value, even if nothing is being written to it. This line is then directly connected to the Rom module.

3.4.5.2 The A/B/C Registers

These registers are meant to store values for calculations within the ALU. They are directly connected to some commands, providing data for calculations. By calling the store (STR) command, the value on the bus at that moment will be stored to the register. The load (LD) command loads the stored value to the data bus in the way described in chapter 3.4.1.2 Output Circuit.

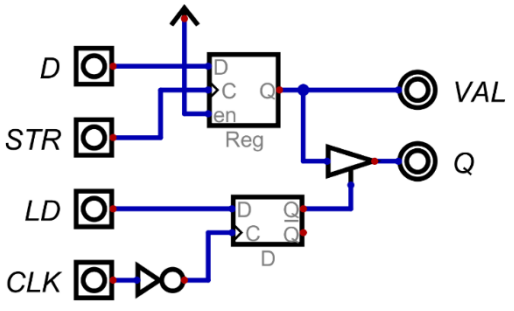


Figure 31: A Simple Register

3.4.5.3 Adder/Subtractor

In this chapter, the function of an adder circuit is explained.

3.4.5.3.1 Negative Numbers

There are multiple approaches to displaying negative numbers in binary. One of these is called “Two’s Complement”. When using this standard, the highest possible value (for example for 8 bits) is cut in half from 255 to 127, with the lowest possible value being -128. This is achieved by making the value of the last bit of a number negative.

binary value				decimal value
0	1	1	1	7
1	0	0	0	-8

Table 11: Bitwise Inverting for 2's Complement

	1	0	0	0	-8
	0	0	0	1	1
+	0	0	0		
	1	0	0	1	-7

Table 12: Adding One for 2's Complement

Creating the negative version of a value is done by inverting every single bit and adding one to the resulting value. Here, the process for inverting the value 7 is shown.

3.4.5.3.2 Addition and Subtraction

Adding binary numbers is similar to adding decimal ones. Usually, the common algorithmic method is used, where overflowing values are carried in the form of small numbers in the next column.

Subtraction is performed in a similar way to Addition, with the only difference lying in the fact that the second value gets inverted with 2’s Complement.

	1	0	0	1
	0	0	1	1
+	0	1	1	
	1	1	0	0

Table 13: Binary Addition

3.4.5.3.3 Carry Out and Overflow

These signals show that the output of the calculation needs more space than is available. Since computers are usually limited in the number of bits available to store values, this can be a problem. In order to detect this, these signals are used. They show, that either the result is unusable or that it needs to be interpreted in a different way.

A carry out signal is generated, when the output is longer than can be stored in a single register. This is usually detected by checking if the last addition still left a bit to be carried. If none of the values used during the addition were negative, the result can still be used under the assumption that the value is now one bit wider. This adds a bit of complexity when working with it.

	1	0	0	1
+	0	0	1	1
1	0	0	1	0
1	1	1	0	0

Table 14: Overflow for 4bit Addition

An Overflow is a bit more complicated. This happens during Subtraction and just shows, that the subtraction does not fit in the provided value. Since 2's Complement is used for subtraction, the last bit of a binary value has a negative value. An overflow means that the addition that comes after the inverting of the second value has crossed into that last bit, corrupting the entire result.

This signal is generated by checking if the last and the previous to last carry bits are different. This check can be done with a single XOR gate.

3.4.5.3.4 Adder Implementation

To implement this behaviour in a logical circuit, the problem needs to be broken down into more manageable pieces. If one was looking at only one slice of an addition table, it could be seen, that it would need three inputs along with two outputs.

The inputs being the two bits of the number to be added and the carry in from the previous column. The outputs would be the bit written at the bottom, and the carry out to the next "slice". This behaviour can now easily be implemented as a circuit. This kind of circuit is called a Full Adder, and since it is meant to behave like a single column of an addition table, it can be stacked in order to create adders of any size by connecting the carry out signals of previous adders to the following ones.

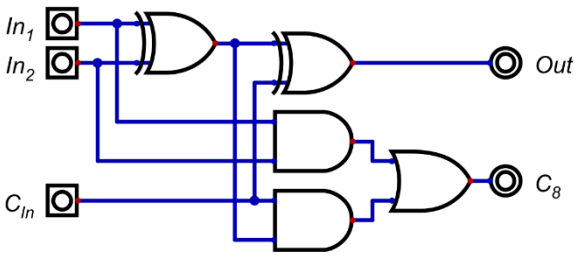


Figure 32: Circuit of a Full Adder

Cin	In1	In2	Out	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 15: Truth Table for a Full Adder

In this table it can be seen that the carry in signal is simply another input, since it does not have any other functionality compared to the other inputs. The output signal has the values for the first digit. The carry out signal is used in order to carry further digits to the next full adder.

There are also Half Adders with a simpler circuit, but they are not able to interact with carry values. However, since they are not used in this computer, they will not be discussed further.

3.4.5.3.5 Subtractor Implementation

The three circuits required in order to implement a subtractor are an inverter, a way to add one to the inverted value, and an adder.

For this, a full adder circuit can be used. Adding one to the inverted value is also easily possible by setting the carry in signal to high, which essentially adds one to the added values. In order to invert the second value however, an actual circuit needs to be implemented. This circuit would need to let values pass through, or invert them, being able to select the behaviour with a single input. The component describing that behaviour is an XOR gate. With one value being low, the other one is directly shown on the output. If it is high however, the output will be the inverted other input.

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Table 16: Truth Table of an XOR Gate

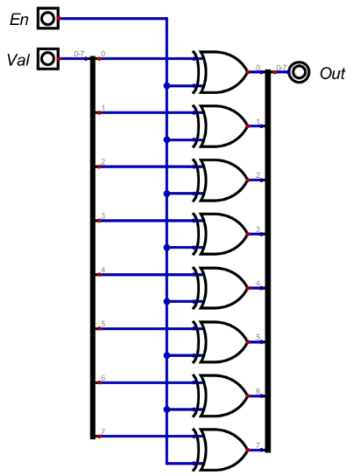


Figure 33: A Toggleable 8Bit Inverter

3.4.5.3.6 Adder Circuit

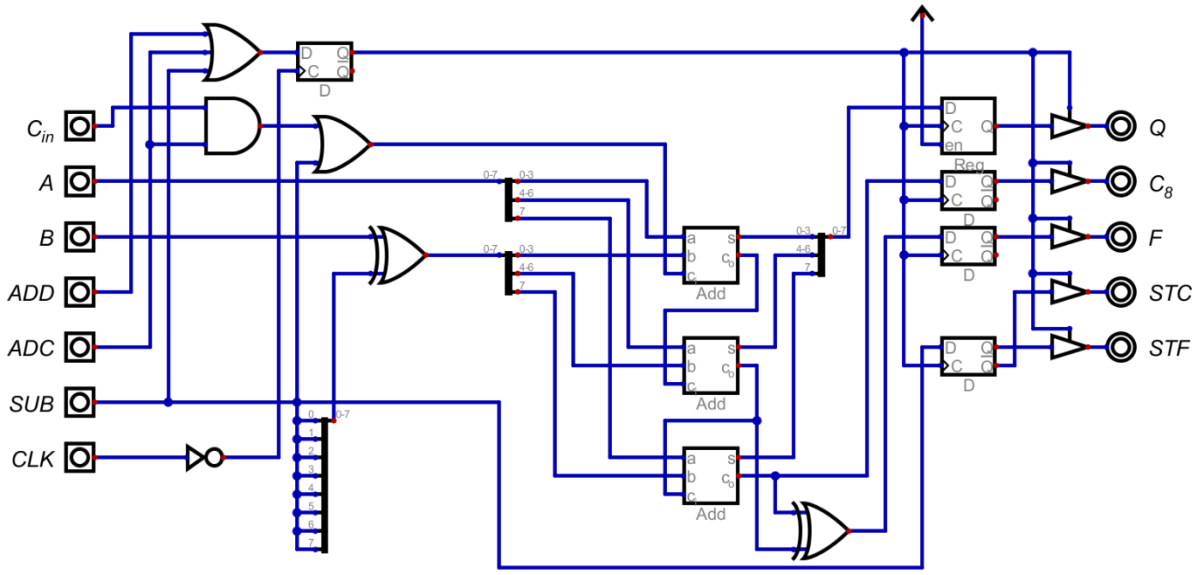


Figure 34: The Adder/Subtractor Circuit

This circuit performs the operations outlined in the chapters 3.4.5.3.4 Adder Implementation and 3.4.5.3.5 Subtractor Implementation. The values upon which it acts are permanently connected to the A and B registers. The commands available to be used are add, subtract, and add with carry, where the adder will take the signal on the carry in input, and use it for the addition.

The triple input OR gate at the top right combines all three inputs, in order to buffer the outputs at the right side of the circuit. The AND gate below will only be high when the carry in input along with the add with carry command are high.

The OR gate to which it is connected will be high if either add with carry is on, or the subtract command is high in order to add one as described earlier. The XOR gate at the input is parallel and inverts the second input as mentioned in chapter 3.4.5.3.5 Subtractor Implementation.

The three adders in the middle of the circuit are there to split the adder in order to get access to the previous to last carry signal needed for overflow detection. This is then done with an XOR gate behind the adder symbols like in chapter 3.4.5.3.3 Carry Out and Overflow.

The STC output is high while outputting values calculated through addition, while the STF output is high when outputting something calculated by subtraction. These control signals are sent to the flag register, where overflow and carry are stored with these signals.

3.4.5.4 Incrementor

This Circuit is meant to increment or decrement the value supplied by the C register by one.

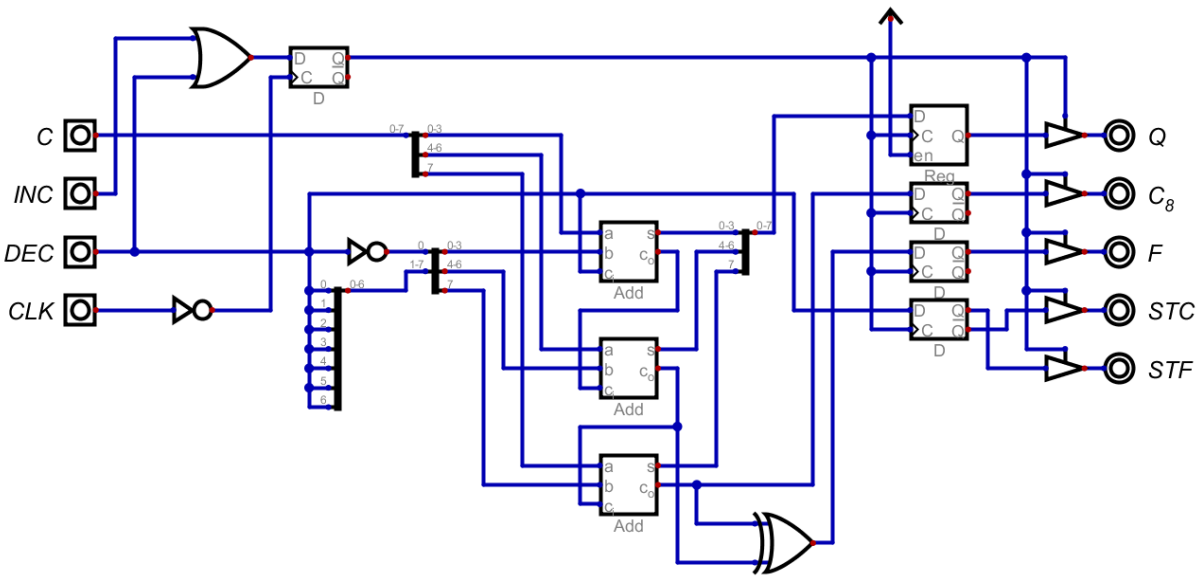


Figure 35: The Incrementor Circuit

In essence, this circuit is the same as the Adder/Subtractor circuit. It is missing one input and the whole inverting circuit needed for subtraction, as the most the input value will change by is one. If either the incrementing or the decrementing input is high, the circuit will perform an addition. If the decrementing input is low during addition, it will be inverted to be one, and sent to the carry in, essentially adding one to the input value. If it is high while performing the operation, the carry in is inverted to be zero, while all the bits of the second input are high. This will add 0xFF to the input, which is negative one if treated as 2's complement.

3.4.5.5 *Flag Register*

This is a specialized register meant to store different states of the computer, here just being the carry and overflow signals from the computer. The carry signal is always outputting to the adder, where it can be used for the add with carry command. The DAT input is connected to the data bus, in order to store values into the register. The STC and STF inputs are connected to the adder module in order to receive signals to store the correct bits. The STA input is the command for storing data to the register. The LDC, LDF, and LDA signals are connected to command decoders and output the carry, overflow, and both flags, respectively. The splitter at the output is just meant to make the two bits work with the eight bits at the output.

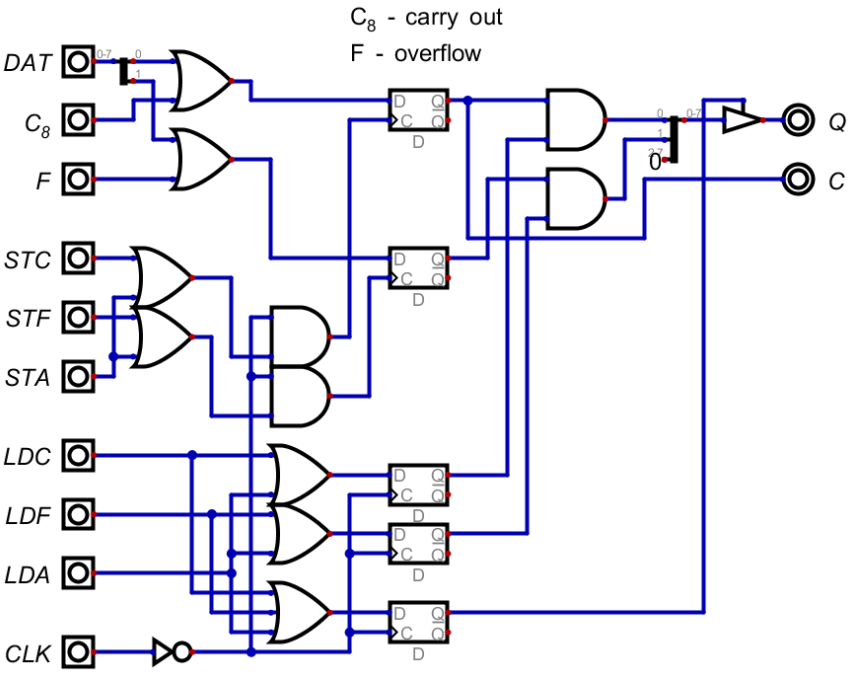


Figure 36: The Flag Register Circuit

3.4.5.6 *Comparator*

This circuit is permanently connected to the A and B registers and continuously compares them. When a command probes for the state of the comparator, it will return 1 when that state is present, and 0 when it is not.

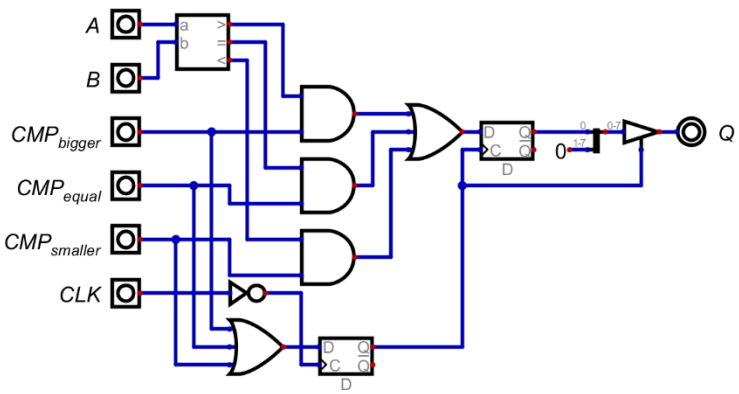


Figure 37: The Comparator Circuit

3.4.5.7 Barrel Shift Register

A shift register is used in order to shift the binary patterns of values, which is needed for certain logical operations. This kind of shift register is used, so that values can be shifted by multiple places within one operation cycle.

3.4.5.7.1 Register

This register stores the distance the shift register shifts the values by, and it is interacted with like any other register in this computer. It also outputs data permanently to the shift register (like in 3.4.5.2 The A/B/C Registers).

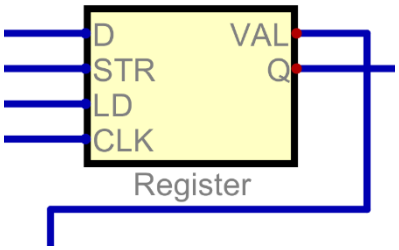


Figure 38: Shift Distance Register

3.4.5.7.2 Shift Circuit

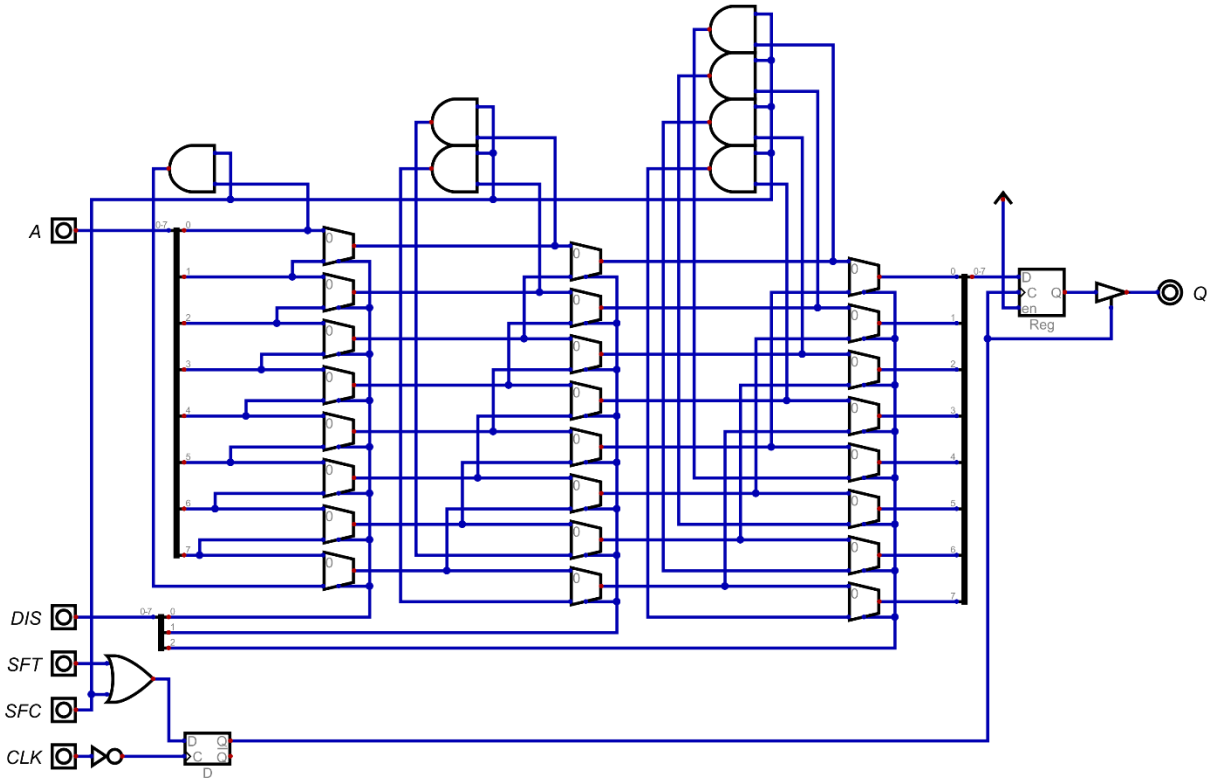


Figure 39: Barrel Shift Circuit

Since the most one could shift an 8-Bit value by is seven positions, there are 3 bits required as a distance input. While the register is 8 bits wide, only the first three are connected to this circuit. The two commands available in this circuit are the normal shift command, where bits that get shifted over the edge are lost, and the circular shift command, which wraps these bits back to the front of the value. Every bit of the distance input is connected to one of the arrays of multiplexers. These multiplexers shift values by powers of 2. The first array shifts them by one, the second one by 2, and the third one by 4. Shifting values is done by hooking up the

input values' places to the second input of the multiplexers at an offset. The first input are the places of the values without being offset. If the multiplexer is set to zero, the unaltered input value is let through, if it is set to one, the shifted value is written to the output. By having a manifold of these shifters, it is possible to shift values by any distance between 0 and 7. The and gates at the top are used for the shift circular command, in order to let the last few values wrap around to the front, only if the shift circular input is high.

3.4.5.8 Binary Logic Operations

In front of the binary operations, there are 8 input OR gates connected to the A and B registers, detecting if there is any value other than zero. If there is, the single bit value given to the logic operations is high, otherwise it is low.

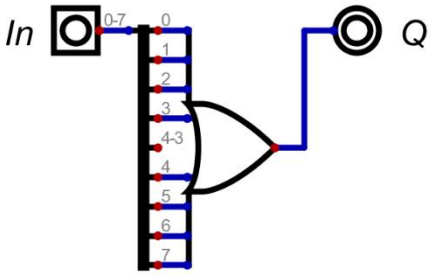
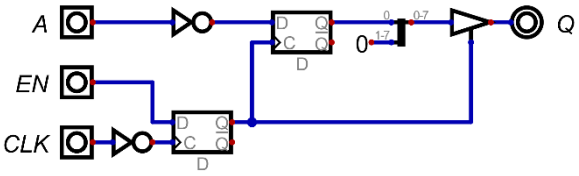


Figure 40: Eight Input OR Gate in Front of Binary Logic Operations

3.4.5.8.1 Not



Since a NOT gate has only one input, the NOT command is only connected to the A value.

Figure 41: Binary NOT Circuit

3.4.5.8.2 Or, And, and Xor

These gates are connected to the binary values of the A and B registers. The only difference between the different circuits is the gate at the input being switched out.

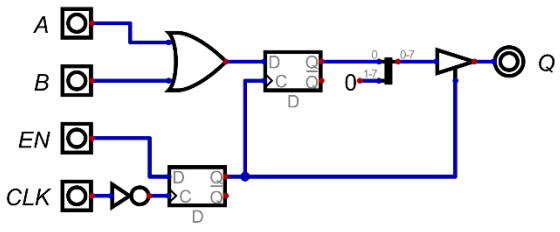


Figure 42: Binary OR Circuit

3.4.5.9 Bitwise Logic Operations

These operations are bitwise, which implies that every bit is treated separately. They are also not connected to the 8 input or gates. Every bitwise circuit has eight of the same logic gates in parallel.

3.4.5.9.1 Not

This command is directly connected to the A register.

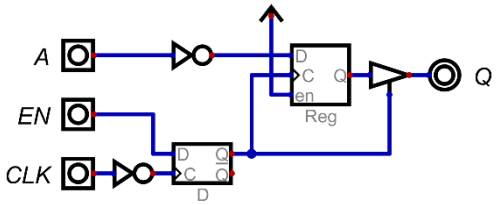
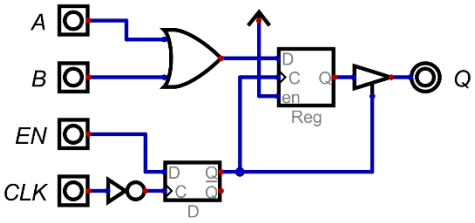


Figure 43: Bitwise NOT Circuit

3.4.5.9.2 Or, And, and Xor



Since they need 2 values, these commands are connected to both the A and B registers.

Figure 44: Bitwise OR Circuit

3.4.6 Input and Output

These simple in- and output circuits were designed so that the computer could be tested and used for simple programmes. One major drawback of these circuits is the need for an understanding of binary values by the user for interaction.

3.4.6.1 Input

The idea behind this input method is to input a value on dipswitches and press a button to accept the value and input it to the computer.

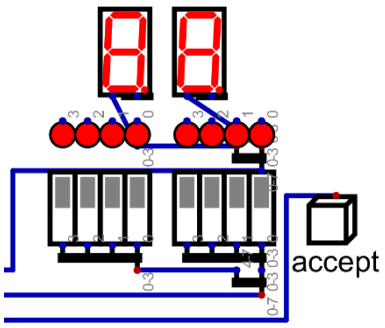


Figure 45: Input Switches

3.4.6.1.1 Input Register

This register is set up similarly to the rest of the registers of this computer (see 3.4.5.2 The A/B/C Registers). This is the place where the value from the input is written to.

3.4.6.1.2 Input Control

This circuit is meant for being checked by the programme, if any new input data has been entered. The inp signal is directly connected to the input accept button, which then sets the value in RS Latch to high. When loading the value from the RS Latch using the output circuit, the output of the D-Type flip flop will be high. Once it is high, it will output values to the bus and when the next falling edge on the clock signal arrives, the RS Latch will be reset. So, the state stored in the RS Latch can only be read once.

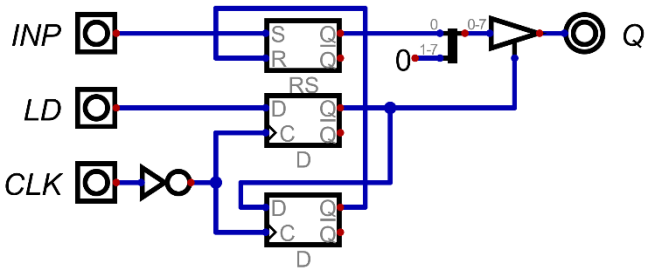


Figure 46: Input Control Circuit

3.4.6.2 Output Register

This is a simple register like the ones in chapter 3.4.5.2 The A/B/C Registers, and it is permanently connected to an array of Leds, showing the state of the register to the user.

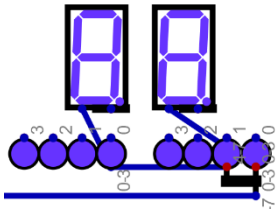


Figure 47: The Display for the Output Register

3.4.7 Not Implemented

While these circuits were not implemented in the real version of this computer, they can still be used in the simulation.

3.4.7.1 D-Pad

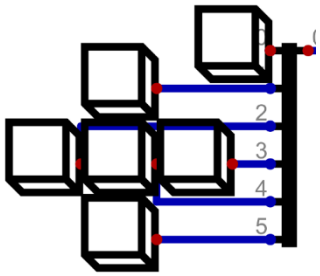


Figure 48: Image of the Input Buttons

This input method was obviously intended for controlling games on the computer, which would help explore its limits. The circuit is simply a register, which stores its values on a falling edge of the clock. This register would only be used in order to buffer values meant to be read by the computer.

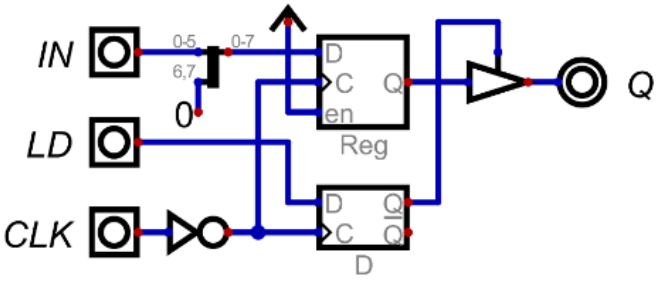


Figure 49: D-Pad Buffer Circuit

3.4.7.2 8 by 8 Pixel Display

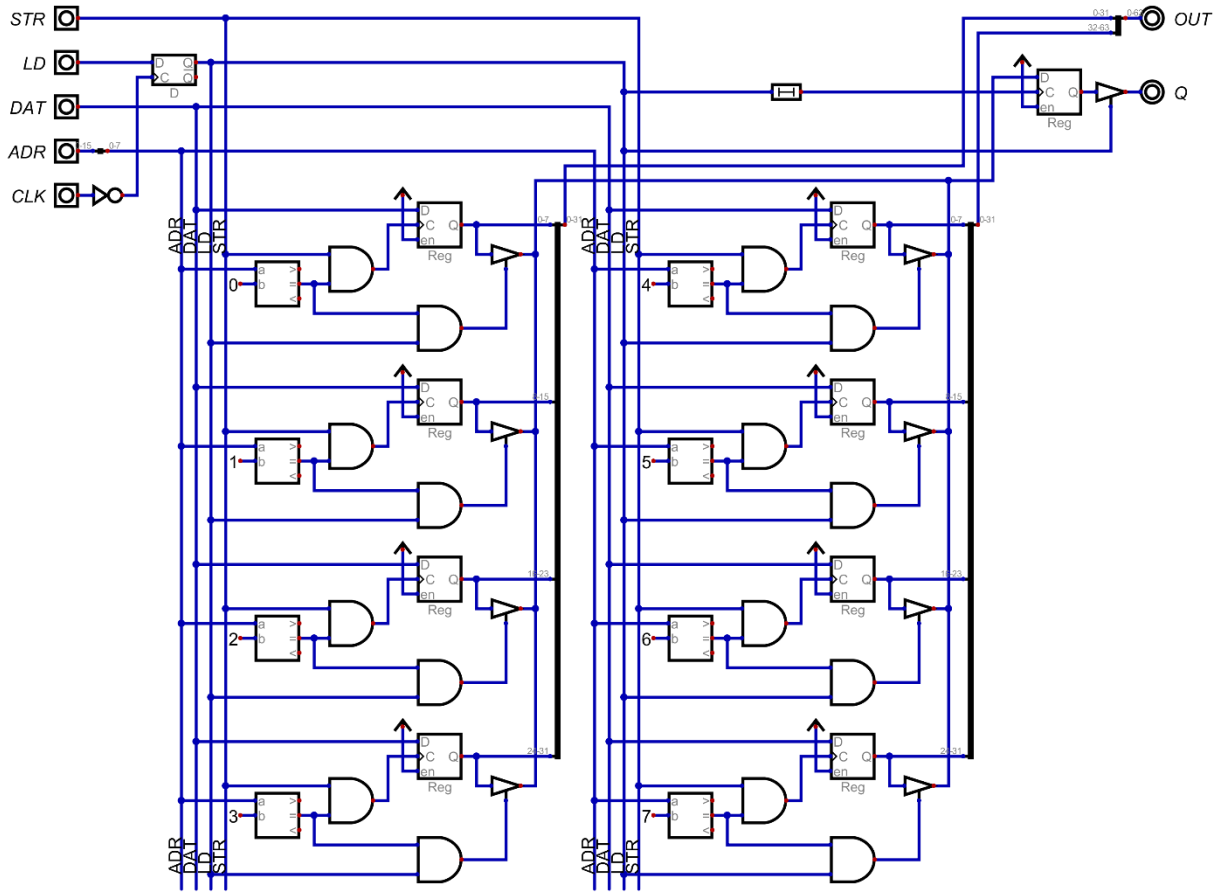


Figure 50: 8 By 8 Pixel Display Logic

This circuit is made up of eight registers, which can be written to and read from. The addresses are resolved by custom address decoders. Every register constantly outputs their values to the “screen”.

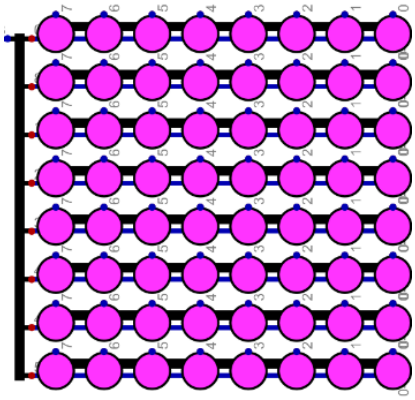


Figure 51: 8 by 8 Pixel Display

4 Hardware – Simon Schmidtgrabner

This part of this diploma thesis focuses on finding the real-world counterparts of the integrated circuits used to simulate the 8Bit-Computer. This involves drawing the schematic plan, tracing the printed circuit board and assembling the components on it, as well as explaining extra circuits, which are not part of the simulation. Furthermore, adding LEDs to every important Bus for better understanding of what every command does and designing an appropriate enclosure for the 8Bit-Computer.

4.1 Components

Most of the ICs used are from Texas Instruments Incorporated, which is a semiconductor manufacturer, and are known by their initial characters SN74, which stands for "Semiconductor Network" and that it belongs to the 7400 series (Wikipedia contributors, 2026). With over 40 different logic subfamilies indicated by different letters after the SN74 and to define the type of IC numbers after the subfamily letters are utilized. An example is:

SN74HC14

The "HC" stands for High-Speed CMOS (Wikipedia contributors, 2026) and the number "14" represents a "hex inverter gate with Schmitt trigger inputs" (Wikipedia contributors, 2026), more about inverters in chapter 3.1.4.1. In addition, the prefix "1G" before the number means that this IC is a one gate version in contrast to the "normal" versions without the "1G" which normally consist of 4 gates in one package.

To find every IC that is needed in the simulation, the database for the "7400-series integrated circuits" (Wikipedia contributors, 2026) on the website Wikipedia was used.

4.1.1 Subfamilies Used

The subfamilies primarily utilized are listed below:

Code	Family	V_{CC}	t_{pd}
74AHC	Advanced High-Speed CMOS	5V ±10%	6.9ns
74LVC	Low-Voltage CMOS	2-3.6V often 5V	3ns
74F	FAST	5V ±5%	3.9ns
74FCT	Fast CMOS	5V ±5%	7ns
74ABT	Advanced BiCMOS	5V ±10%	3.6ns
74HCS	Schmitt-Trigger Integrated High-Speed CMOS	2-6V	13ns

Table 17: List of used IC-Subfamilies

Information taken from: (Wikipedia contributors, 2026)

4.1.2 ICs Used

The Boolean Logic ICs are displayed in the following table:

Chip Name	Chip Function	Gate amount	Inputs/ Gate	Use Case	Explanation	Input
SN74AHC14	NOT	6	6	Inverter	3.1.4.1	Stt
SN74LVC1G14	NOT	1	1	Inverter		Stt
SN74LVC1G04	NOT	1	1	Oscillator		Unb
SN74AHC08	AND	4	2	AND	3.1.4.2	
SN74LVC1G08	AND	1	1	AND		
SN74AHC32	OR	4	2	OR	3.1.4.3	
SN74HCS4075	OR	3	3	8 in OR		Stt
SN74LVC1G32	OR	1	1	8 in OR		
SN74AHC86	XOR	4	2	XOR	3.1.4.4	
SN74LVC1G86	XOR	1	1	XOR		

Table 18: Boolean Logic ICs

The General Logic ICs are displayed in the following table:

Chip Name	Chip Function	Bit Size	Inputs/ Gate	Use Case	Explanation Chapter
SN74F521	Identity comparator	8	16	CMD decoder	3.4.1.1
SN74HC682	Magnitude comparator	8	16	Comparator	3.4.5.6
SN74F283	Full adder	4		Adder, Subtractor	3.4.5.3.2
CY74FCT163	Binary Counter	4		Programme Counter	3.4.4.1
LM555	Timer			555-Timer clock	4.3.2.3

Table 19: General Logic ICs

The Storage Management ICs are displayed in the following table:

Chip Name	Chip Function	Bit Size	Words	Use Case	Explanation Chapter	Output
AS7C1024B-12TCN	SRAM	8	128K	RAM	3.4.3.2	
M29F800FT	NOR Flash	16	512K	ROM	3.2.2 3.4.3.1	
SN74ABT574	flip-flops	8		Buffer/ Register	3.1.5.1 3.2.4	Tristate
SN74ABT245B	Driver	8		Bus Driver	3.3.3.3	Tristate
SN74AHC74	D-latch	2		D-Latch SR-Latch	3.1.5.1	Open-Drain
SN74AHC157	Multiplexer	2->1		Barrel Shift Register	3.4.5.7	Open-Drain

Table 20: Storage Management ICs

The Input / Output components are displayed in the following table:

Name	Function	Bit Size/ Version	Inputs / Gate	Use Case	Explanation Chapter
SN74AHC549	NOT	1	8	LED Driver	3.4.1.1
LTST-C170KFKT	LED	Orange	1	Indicator CMD Bus	
LTST-C170KGKT	LED	Green	1	Indicator ADR Bus	
LTST-C170KRKT	LED	Red	1	Indicator Output/ DAT Bus	
LTST-C171KSKT	LED	Yellow	1	Indicator LN Bus	
LTST-C171TBKT	LED	Blue	1	Indicator Input Bus	
B3FS-4055P	Push Button	OFF - (ON)	1	Button CLK	4.3.2.2
M61P104KT20T607	Potentiometer	100kΩ		555-Timer R_P	4.3.2.3
12D00G	Slide Switch	ON-ON		I/O, CLK Select	

Table 21: Input/ Output components

The Other components are displayed in the following table:

Name	Function	Bit Size/ Version	Use Case	Explanation Chapter
1N4148WS-7-F	Diode	1	DAT/ ADR disable	
TMUX1309DYYR	Multiplexer	8:1 /4:2	CLK Multiplexer	4.4.2.5
Adafruit 5807	USB C PD Trigger	20V	Computer PSU	4.3.3
DS1100Z-150+	delay	150ns	CLK delay	
DS1100Z-20+	delay	20ns	Delay DAT/ ADR disable	
MP2338GTL	Step-Down Converter	20V->5V	DC-DC Converter from 20V to 5V	4.3.4
PISM-6R8M-04	Power-Inductor	4.4A	For DC-DC Converter	4.3.4
296-1-010-0-S-RT0-1159	Pin Header	10pin	Connection between PCBs	
ML2585X2-120	Pin Socket	20pin	Connection between PCBs	

Table 22: Other components

4.2 Programming device

To program the parallel flash memories, a programming device with related software is needed. There were multiple options for a programmer, which will be compared in the following.

Model	XGecu T48	XGecu T56	XGecu T76
Device Support (Parallel NOR Flash)	✓ (limited vs. newer chips)	✓ (broader support incl. many parallel Flash)	✓ (widest support)
SPI-Flash-Support	✓	✓	✓
NAND Support	✗	✗	✓
MCU Support (direct microcontroller programming)	✗ /limited	✗	✓
Pin Count Supported	Up to ~48	Up to ~64	Up to ~76 / larger adapters
Adapter Types Support	Minimal	Better adapter pack	Best adapter pack
Speed	Good	Faster	Fastest (better hardware)
Software	Basic	Improved	Most polished
Price	~50€	~80€ (bad availability)	~140€

Table 23: Comparison of Programmers

From the comparison table it is evident that the XGecu T76 is the newest and the best programmer with the widest range of chip support, not only for parallel flash memories but also for NAND flash, MCU and SPI-programmable flash. The **XGecu T76** was chosen not only for use in this diploma thesis but also to be available for future diploma theses as a programmer.

4.3 Circuit diagram planning – Simulation to Hardware implementation

For simulated circuits to work in hardware, some additional circuitry is needed, for example, to program the two flash memories or to implement different clocks with diverse frequencies.

4.3.1 Flash memory programming circuitry

To ensure that the flash programmer Xgecu T76 is able to programme the two NOR parallel flash memories correctly, additional circuitry is needed to disconnect specific flash memory pins from defined states, such as a pull-down to ground or a connection to any sort of bus on the PCB. This is done via multiple bus driver chips, for example, the SN74ABT245B. The exact circuitry will be outlined in more detail in chapter 4.4.2.1.

4.3.2 Clock generators

A clock generator in simulation is one simple component, but in hardware consists of many different components to work properly. In the following section, different clock generators for this computer will be further explained.

4.3.2.1 Pierce Oscillator

The Pierce Oscillator, named after its inventor George W. Pierce, is a circuitry consisting of a quartz crystal X, two capacitors C_1/C_2, two resistors R_1 / R_2 and an inverter chip U_1, like the SN74LVC1GU04, which is specifically built for this kind of use. Additionally, an inverter chip with Schmitt-trigger inputs U_2, like the SN74LVC1G14, can be connected and used as the circuit's output for a better rectangular clock signal.

The quartz crystal X is used in its "parallel resonant" mode. When the circuit starts up, ambient noise provides a tiny bit of energy. The crystal acts like a very narrow bandpass filter, only allowing a specific frequency to pass through. It behaves like an inductor in this configuration.

The inverter U1 contains an unbuffered output and is commonly used in oscillator circuits due to having lower total gain than its buffered equivalent. Here it is used specifically to invert its input signal from 0° to 180°. However, for oscillation to be sustained, a total phase shift of 360° around the loop is needed.

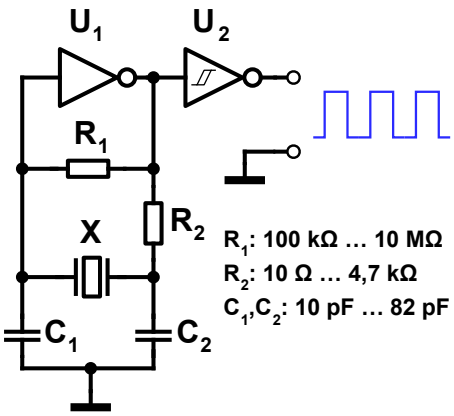


Figure 52: Pierce Quartz Oscillator

(Wdwd, Stündle, 2011)

To get the remaining 180° of phase shift of the total 360°, the circuit uses a Pi network consisting of the crystal and the two load capacitors C_1 and C_2. The crystal's inductance combined with the two capacitors creates a phase lag. By the time the signal travels from the output of the inverter, through the crystal network, and back to the input, it has been shifted 180°. Combining the 180° of the inverter and the 180° of the Pi network, the total needed 360° shift for the oscillation is achieved.

The resistor R_1 secures the linear operation of the inverter U_1, while the resistor R_2 is limiting the peak current when the output resistance of the inverter U_1 is not high enough. Both of these resistors can be left out when not needed; if so, R_2 can be replaced with a direct connection and R_1 with an interruption of the circuit.

In this diploma thesis, these values for the Pierce oscillator were used:

$$R_1 = 330\Omega$$
$$R_2 = 1M\Omega$$
$$C_1 = C_2 = 27pF$$
$$X = 4MHz$$

The values for C_1 and C_2 can be found in the datasheet for the quartz crystals but are mostly around 20pF.

4.3.2.2 Button clock

As the name suggests, a button clock is nothing more than a button which will be pressed to generate a rising edge, and when released, a falling edge; in summary, a clock signal for one program cycle will be generated.

4.3.2.3 555-Timer clock

A 555-Timer clock is using a 555 Timer with some additional circuitry to generate a specific range of clock frequencies, in our case 1 Hz to 100 Hz, to better demonstrate the different program cycles without the need of pressing a button and to still be able to track the various unique operations occurring during these cycles. As the 555 Timer has 2 different modes, monostable and astable, we used the astable mode for the clock generation.

The following figure, taken from (Texas Instruments, p. 10), is showing the circuit for the astable mode.

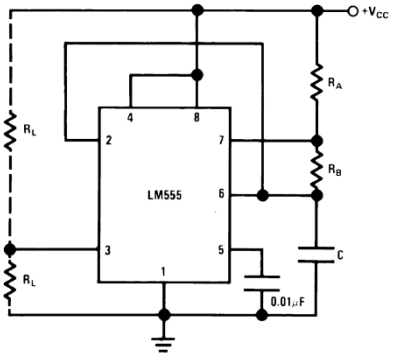


Figure 53: 555-Timer Astable

(Texas Instruments, p. 10)

4.3.2.3.1 Duty cycle

The resistors R_A and R_B are used to charge the capacitor C, while only the resistor R_B is used for discharging C. Because of that, the duty cycle for the clock signal is set by the ratio of these two resistors. The duty cycle D is calculated with this formula:

$$D = \frac{R_B}{R_A + 2 \cdot R_B}$$

Equation 1: 555-Timer duty cycle

(Texas Instruments, p. 11)

4.3.2.3.2 Frequency

In the astable mode the capacitor C charges and discharges between 1/3 V_{CC} and 2/3 V_{CC}. Due to this, the formula for calculating the generated clock frequency is:

$$f = \frac{1}{T} = \frac{1}{(R_A + 2 \cdot R_B) \cdot C}$$

Equation 2: 555-Timer frequency

(Texas Instruments, p. 11)

For the specific range of 1Hz to 100Hz, a potentiometer is needed. It is connected in parallel with R_B to change the frequency with it. A resistor is connected in series to the potentiometer to ensure the parallel circuit of the potentiometer and R_B is never 0Ω when the potentiometer is turned to its lowest resistance value of 0Ω.

4.3.2.3.3 Duty cycle and Frequency calculation

To achieve a duty cycle of nearly 50% it is evident that in the *Equation 1: 555-Timer duty cycle* R_A needs to be chosen as small as possible to get a ½ ratio with R_B. The smallest value that RA can be can be calculated on the basis of Ohm's Law, the information that the maximum Power dissipation P_{dp} of the resistors we are using with the model number CR1206 is 250mW (Bourns, REV. 03/21, p. 1) and that V_{CC} is 5V.

$$R_A = \frac{(V_{CC})^2}{P_{dp}} = \frac{(5V)^2}{250mW} = 100\Omega$$

Equation 3: 555-Timer Ohm's Law

To calculate R_B, *Equation 1: 555-Timer duty cycle* can be transformed to solve for R_B but because R_A is 100Ω R_B needs to be larger than ~800kΩ to guarantee a duty cycle of nearly 50%. But to be sure, a 1MΩ resistor was taken. Now if we put the potentiometer and its serial resistance to R_B in parallel the frequency can be changed on the cost of the duty cycle. The serial resistance R_{sP} of the potentiometer will be assumed to be 1.5kΩ. To calculate the capacitance C the *Equation 2: 555-Timer frequency* can be transformed to C. The resistance of the potentiometer R_p will be zero and only the serial resistance R_{sP} will be in parallel with the 1MΩ of R_B. The frequency f_{max} with 100Hz will be used to calculate C.

$$C = \frac{1}{f_{max} \cdot \left[0.693 \cdot (R_A + 2 \cdot \frac{R_B \cdot (R_P + R_{sP})}{R_B + R_P + R_{sP}}\right]} = \frac{1}{100Hz \cdot \left[0.693 \cdot (100\Omega + 2 \cdot \frac{1M\Omega \cdot (0\Omega + 1.5k\Omega)}{1M\Omega + 0\Omega +}\right]} = 4.662\mu F$$

Equation 4: 555-Timer Capacitance calculation

The next closest to C of 4.662µF in the E24 (Wikipedia contributors, 3 March 2026) is 4.7µF.

With the assumed 1.5kΩ for R_{sP} the Duty cycle for f_{max} = 100Hz can be calculated:

$$D_{fmax} = \frac{\frac{R_B \cdot (R_P + R_{sP})}{R_B + R_P + R_{sP}}}{R_A + 2 \cdot \frac{R_B \cdot (R_P + R_{sP})}{R_B + R_P + R_{sP}}} = \frac{\frac{1M\Omega \cdot (0\Omega + 1.5k\Omega)}{1M\Omega + 0\Omega + 1.5k\Omega}}{100\Omega + 2 \cdot \frac{1M\Omega \cdot (0\Omega + 1.5k\Omega)}{1M\Omega + 0\Omega + 1.5k\Omega}} = 0.484$$

Equation 5: 555-Timer Duty cycle fmax

To calculate the resistance of the potentiometer R_p the *Equation 2: 555-Timer frequency* is transformed to R_p and f_{min} with 1Hz is inserted.

$$R_p = \frac{(R_B+R_{sP})\cdot\left(\frac{1}{2\cdot 0.693\cdot C\cdot f_{min}}\right) - R_B\cdot R_{sP}}{R_B+R_A-\frac{1}{2\cdot 0.693\cdot C\cdot f_{min}}} = \frac{(1M\Omega+1.5k\Omega)\cdot\left(\frac{1}{2\cdot 0.693\cdot 4.7\mu F\cdot 1Hz}\right) - 1M\Omega\cdot 1.5k\Omega}{1M\Omega+100\Omega-\frac{1}{2\cdot 0.693\cdot 4.7\mu F\cdot 1Hz}} = 179.71k\Omega$$

Equation 6: 555-Timer potentiometer resistance Rp

The potentiometer resistance R_p can be in the range of 100kΩ to 200kΩ, a value of 100kΩ for the potentiometer was selected with that the maximum and minimum frequency and the duty cycle D_{fmin} for f_{min} is as follows:

$$f_{min} = \frac{1}{0.693\cdot\left[R_A+\frac{R_B\cdot(R_P+R_{sP})}{R_B+R_P+R_{sP}}\right]\cdot C} = \frac{1}{0.693\cdot\left[100\Omega+\frac{1M\Omega\cdot(100k\Omega+1.5k\Omega)}{1M\Omega+100k\Omega+1.5k\Omega}\right]\cdot 4.7\mu F} = 1.665Hz$$

Equation 7: 555-Timer f_{min}

$$f_{max} = \frac{1}{0.693\cdot\left[R_A+\frac{R_B\cdot(R_P+R_{sP})}{R_B+R_P+R_{sP}}\right]\cdot C} = \frac{1}{0.693\cdot\left[100\Omega+\frac{1M\Omega\cdot(0\Omega+1.5k\Omega)}{1M\Omega+0\Omega+1.5k\Omega}\right]\cdot 4.7\mu F} = 99.183Hz$$

Equation 8: 555-Timer f_{max}

$$D_{fmin} = \frac{\frac{R_B \cdot (R_P + R_{sP})}{R_B + R_P + R_{sP}}}{R_A + 2 \cdot \frac{R_B \cdot (R_P + R_{sP})}{R_B + R_P + R_{sP}}} = \frac{\frac{1M\Omega \cdot (100k\Omega + 1.5k\Omega)}{1M\Omega + 100k\Omega + 1.5k\Omega}}{100\Omega + 2 \cdot \frac{1M\Omega \cdot (100k\Omega + 1.5k\Omega)}{1M\Omega + 100k\Omega + 1.5k\Omega}} \approx 0.5$$

Equation 9: 555-Timer D_{fmin}

4.3.3 8Bit-Computer Power Supply

To supply power to the PCBs the" Adafruit USB Type C Power Delivery Dummy Breakout - I2C or Fixed - HUSB238" (Adafruit Industries) from Adafruit was used.

For adjusting the output voltage and maximum current of the breakout board the connection pads of "5V" and "1A" were interrupted via cutting them to provide the needed 20V and 5A.

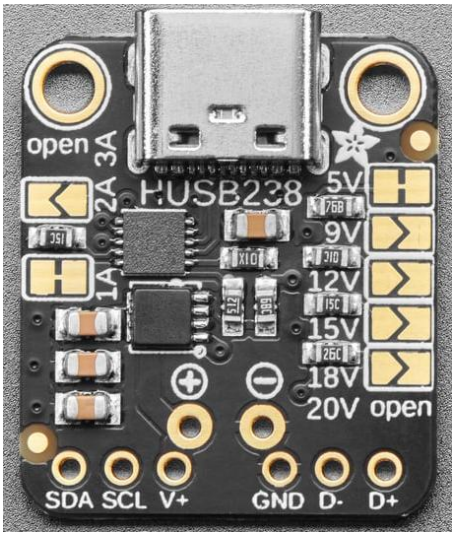


Figure 54: Adafruit USB Type C Power Delivery Breakout (Adafruit Industries)

4.3.4 DC-DC Step-down converter

With the provided 20V from the PD Source a DC-DC Converter is needed for the wanted 5V, which is necessary for every IC and Component on the PCBs.

In this diploma thesis the MP2338GTL which is a "[…] fully integrated, high-frequency, synchronous, rectified, step-down switch-mode converter with internal power MOSFETs." (Monolithic Power, 12/11/2020, p. 1) used for converting the 20V to 5V.

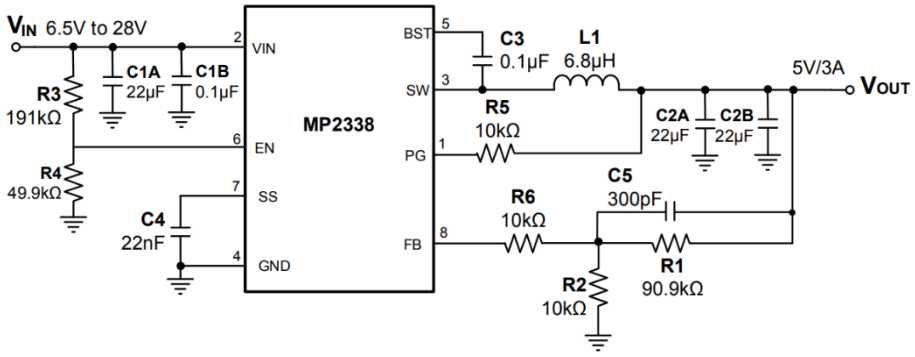


Figure 55:DC-DC Converter 20V to 5V, 3A (Monolithic Power, 12/11/2020, p. 20)

4.4 Schematic plan

To draw a schematic plan and the corresponding printed circuit board CAD software is needed.

4.4.1 KiCad

In this diploma the Open-Source PCB Design suite KiCad was used because of its unrestricted pcb board size and layers as well as the strong community and IC-Manufacturer support.

4.4.2 Drawing of the schematic plan

While drawing the schematic plan, multiple sheets were used and were included in the root file via “Draw Hierarchical Sheets”. Furthermore, Net Classes were used to change the colours of the wires and buses for a better clarity.

4.4.2.1 Flash Memory circuitry

4.4.2.1.1 Programming circuitry

For the Flash Memories able to be programmed, specific circuitry is needed to interrupt the connection between the Flash pins and the Buses to have no short circuits. For the interruption, the Bus driver ICs SN74ABT245B are used.

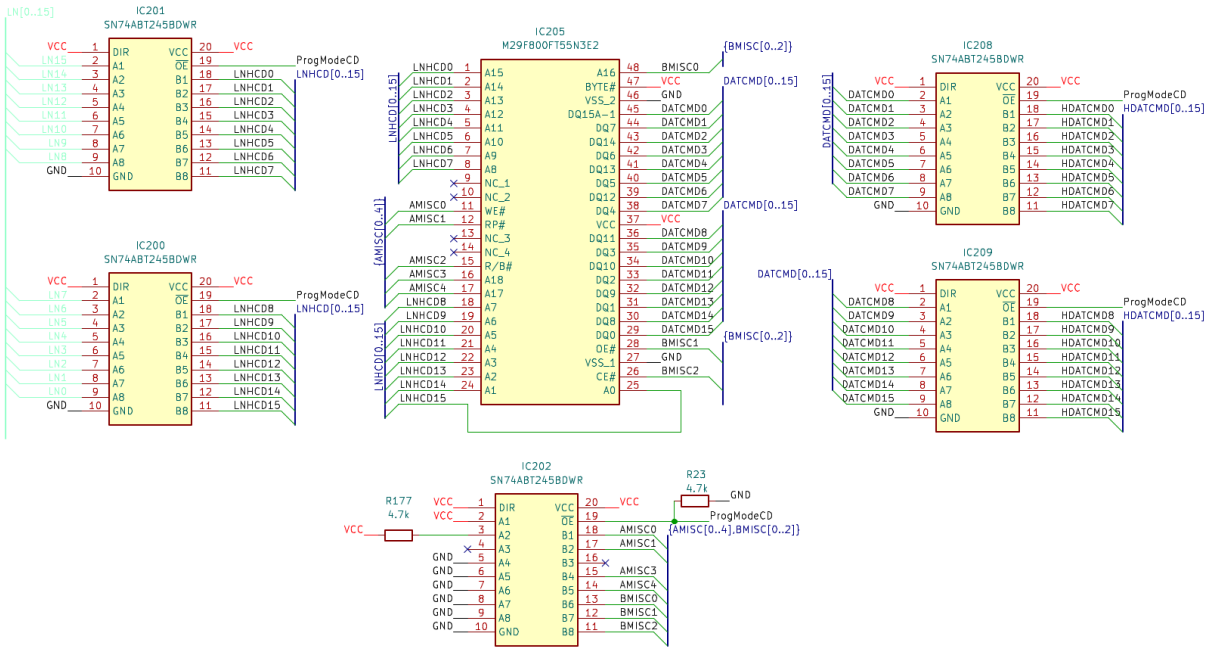


Figure 56:Flash Memory program circuit

When the programming cable is plugged into the CMD/DAT socket the wire ProgModeCD gets bridged to 5V and all the Bus Drivers switch their outputs to High Impedance Mode for the Flash to be programmed, normally the pull-down resistor R23 is pulling the ProgModeCD wire to Ground. As from the figure seen the wires AMISC0 to AMISC4 and BMISC0 to BMISC2 are normally pulled down either to the Ground or VCC potential. When in programming mode all wires to and from the Flash are in floating state so that the programmer can pull them to their needed potentials so a program can be written to it.

AMISC2 is an exception and is left floating in both programming and operation mode because it is an open drain Output from the Flash which is indicating if something is written to it via programming.

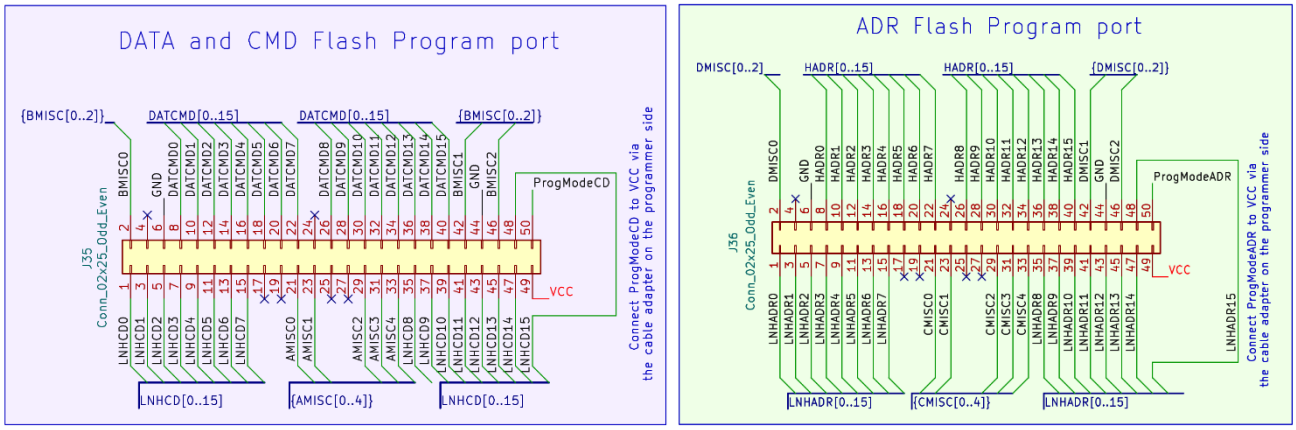


Figure 57: Flash Programming Ports

4.4.2.1.2 DAT and ADR disable Circuitry

To deactivate the DAT and ADR Bus when writing from something else than the Flash Memories the Bus driver ICs SN74ABT245B are used.

The ADR_{dis} signal is directly coming from the Data to Address (see chapter3.4.4.2) and deactivates the Drivers to use the Data Bus for reading from the RAM via the Addresses as well as writing to the RAM. The Data to Address Interface circuitry will be explained more in detail in chapter 4.4.2.4.

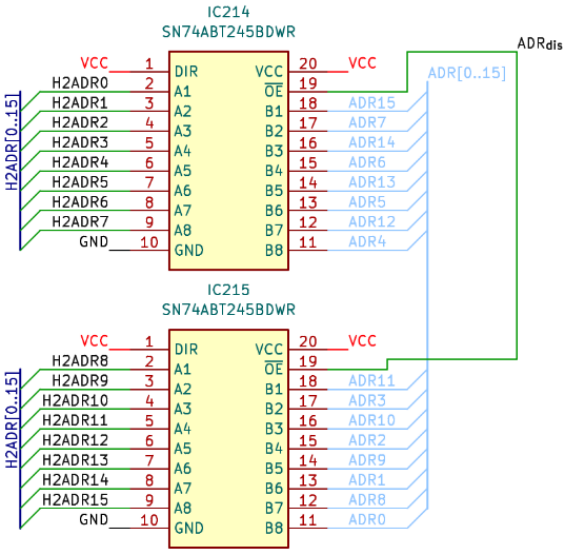


Figure 58: Flash ADR_{dis}

The DAT_{dis} circuit is slightly more complex because of the extra 16ns delay on the $\overline{CLK}$ via the DS1100Z-20+ IC and the D-Latch SN74AHC74DR which is shifting the DAT_{dis} for half of a $\overline{CLK}$ cycle, explained why in chapter 3.4.3.1

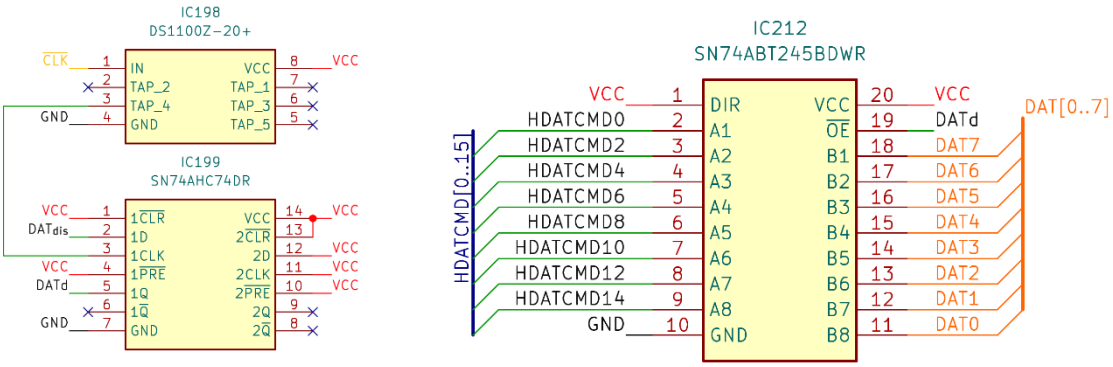


Figure 59: Flash DAT_{dis}

4.4.2.2 Register circuitry

In the following the circuitry for a Register, explained in chapter 3.4.5.2, will be shown.

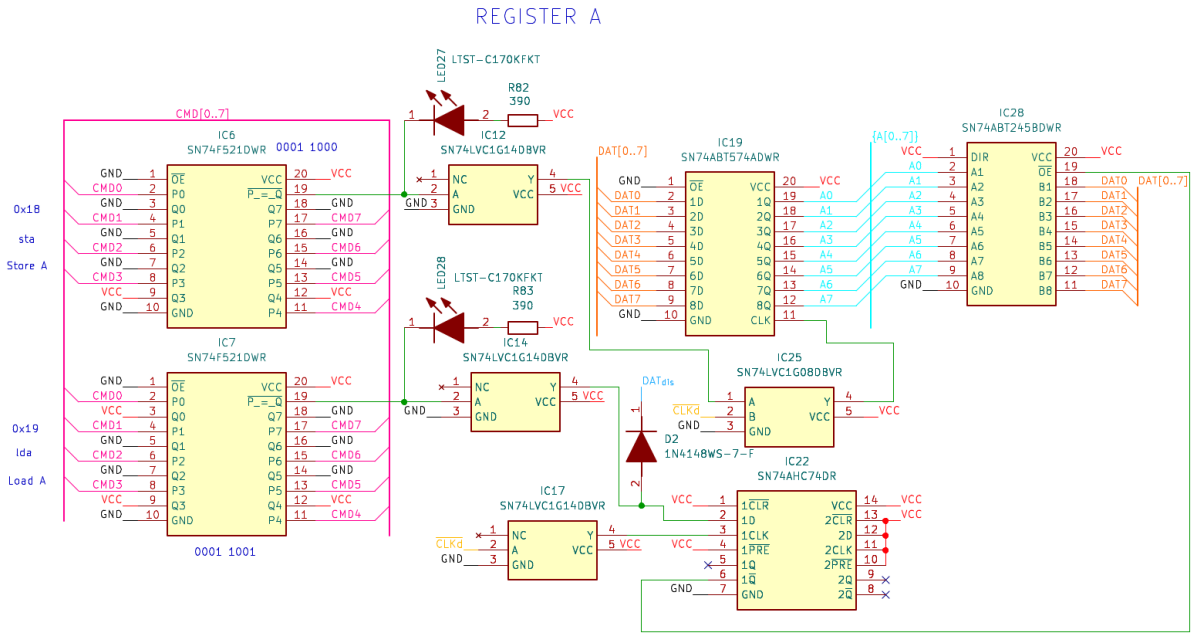


Figure 60: Hardware Register A

Because of the normally inverted Output of the CMD decoder, an inverter like the SN74LVC1G08 will be needed for the Output to be inverted back from low active to high active. This inverter is needed for every CMD decoder used in the 8Bit-Computer.

4.4.2.3 *RAM circuitry*

For the RAM to have the right timing for saving data, extra circuitry is needed.

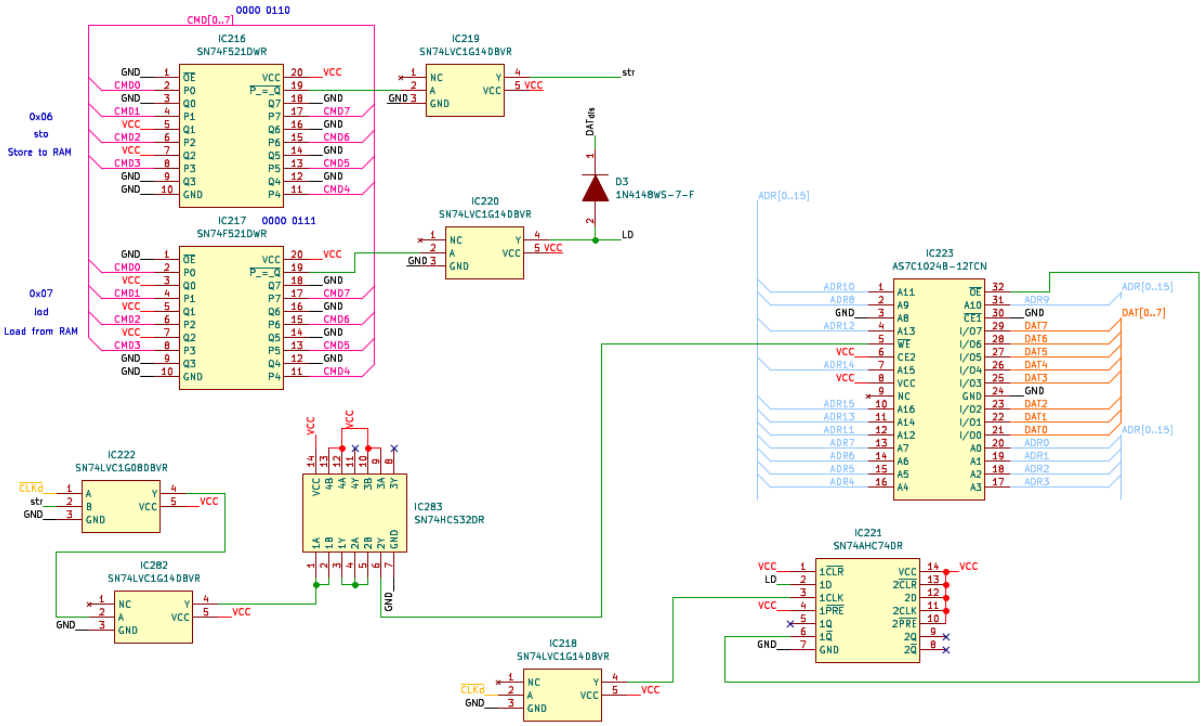


Figure 61: RAM circuitry

Because the RAM is finishing its write cycle when there is a transition from 0V to 5V on the $\overline{\text{WE}}$ pin of the RAM, an AND gate was used to initiate that transition on the falling edge of the $\overline{\text{CLKd}}$ signal. With that the RAM can save the data on the DAT bus in the first half of a clock cycle compared to one full cycle because of CMD only transitioning after one full clock cycle. In addition, the address needs to be applied for the whole write process to be successful.

4.4.2.4 Data to Address Interface circuitry

To save data in the RAM without the Addresses from the Flash the Data to Address Interface is needed, further explained in chapter 3.4.4.2..**Fehler! Verweisquelle konnte nicht gefunden werden.**

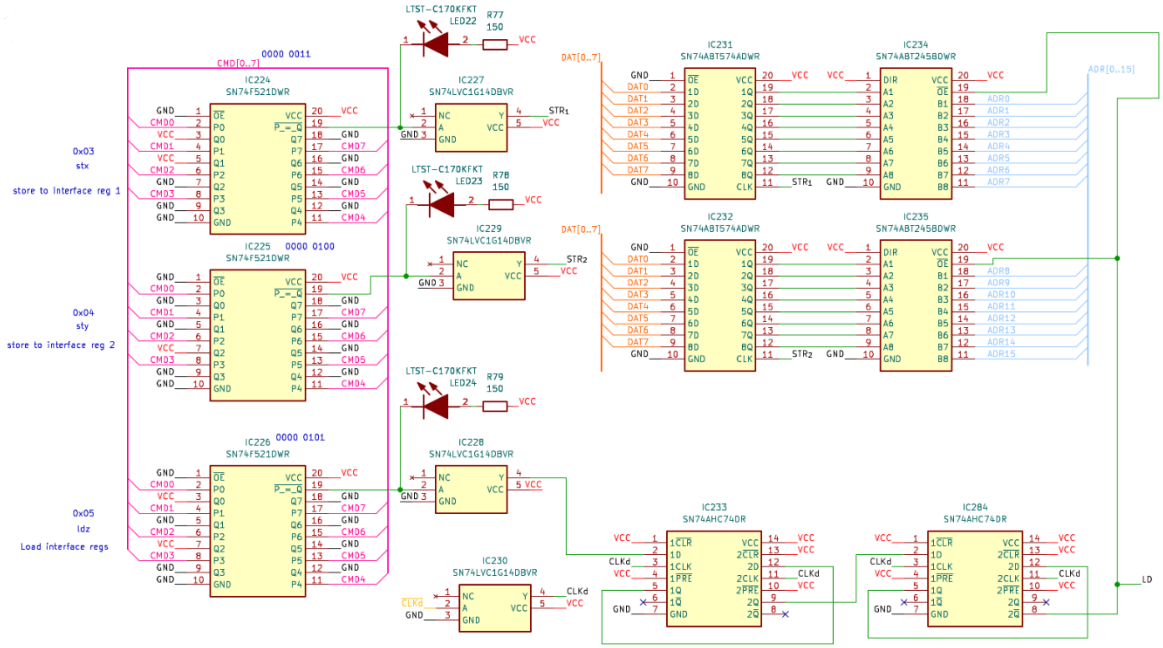


Figure 62: Hardware Data to Address Interface

The LD wires use cases are to deactivate the Address Output of the Flash memory and to activate the Output of the saved Address in the Data to Address Interface. It is essential to delay the Output of the Interface for two complete cycles compared to the CMD to be able to read from the RAM because the RAM needs the Address applied for the whole reading process to work. Accomplished here with four D-latches with the non-inverted Output of the first Latch inputted to the D input of the second Latch and so on. All four D-Latches are using the same Clock.

4.4.2.5 Clock implementation

4.4.2.5.1 Different Clocks

The following shows how the different Clock signals from all the generators were connected to switch between them.

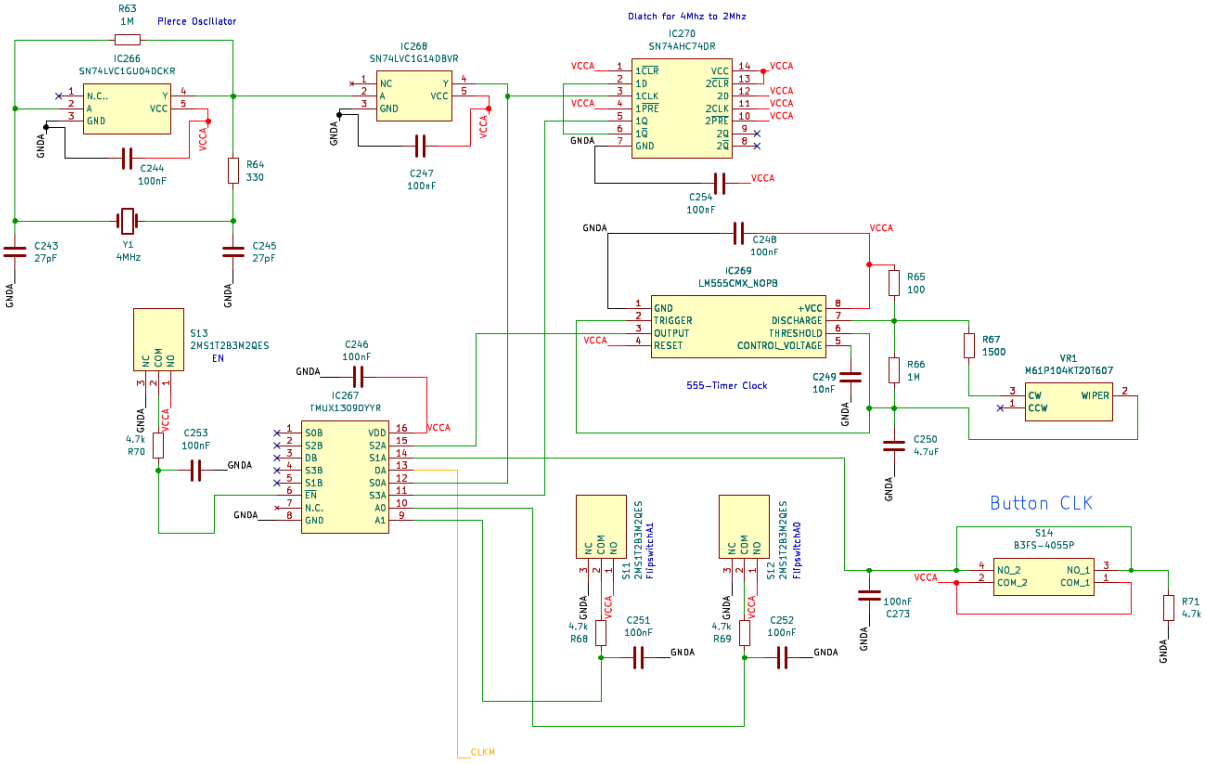


Figure 63: Hardware Clock implementation

In the top left corner is the Pierce Oscillator (see 4.3.2.1) and in the top middle is a D-Latch wired to divide the Clock signal coming from the Pierce Oscillator from 4MHz to 2MHz.

The middle circuitry is the 555-Timer and its components for the astable mode (see 4.3.2.3). The potentiometer is connected on its CW and WIPER input to define the turn direction to clockwise for increasing and counterclockwise for decreasing the frequency.

In the bottom-right part, the Button clock (see 4.3.2.2) can be seen. A pull-down resistor and a button debounce capacitor were used.

The 2MS1T2B3M2QES are placeholder switches for the 12D00G Slide Switches.

To change the Clock signal source, the multiplexer TMUX1309 was used. With the EN Flip switch the Clock can be completely turned off. The Flip switches A0 and A1 are controlling the Clock source corresponding to the following truth table.

A1	A0	Clock Source
0	0	Pierce Oscillator 4Mz
0	1	555-Timer 1Hz to 100Hz
1	0	Button Clock
1	1	Pierce Oscillator 2Mz

Table 24: Clock Multiplexer Truth Table

For every Switch and Button, a RC circuit was utilized to debounce and limit the current when switching.

4.4.2.5.2 Delayed Clock

To guarantee that the Flash has enough time to write its CMDs, DATA and ADRs the Clock wire to all other circuitry except the Flash and Programme Counter needs to be delayed by around 85ns. For this the DS1100Z-150+ was used. This IC delays every signal exactly 30ns, 60ns, 90ns, 120ns or 150ns, the 90ns option was used to leave a little headroom.

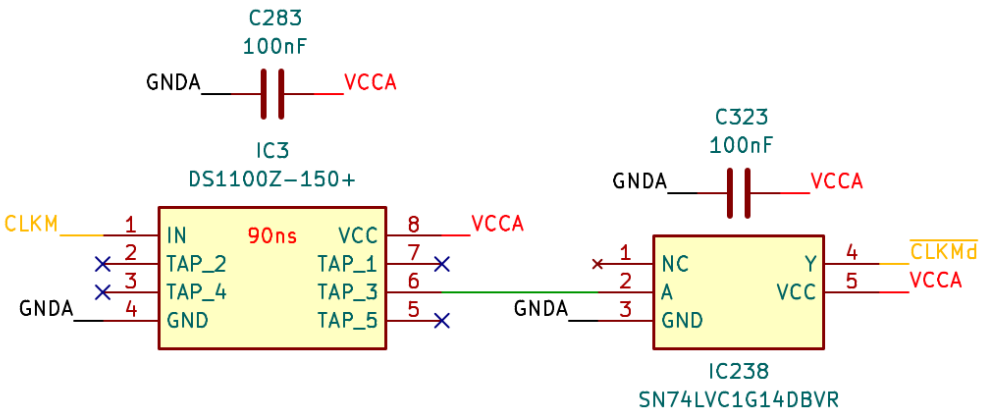


Figure 64: Hardware Delayed and Inverted Clock

The 90ns delayed Clock is then inverted and wired to all circuitries. $\overline{CLKMd}$ is identical to $\overline{CLKd}$ but under a different name. The same applies to CLKM which is also seen in the Programme Counter and Flash memory circuitry under $\overline{CLK}$.

4.4.2.6 Bus LED indication circuit

To better visualise what is happening at the current Programme line, LEDs were used to indicate every Bit for every important Bus for example the DAT bus.

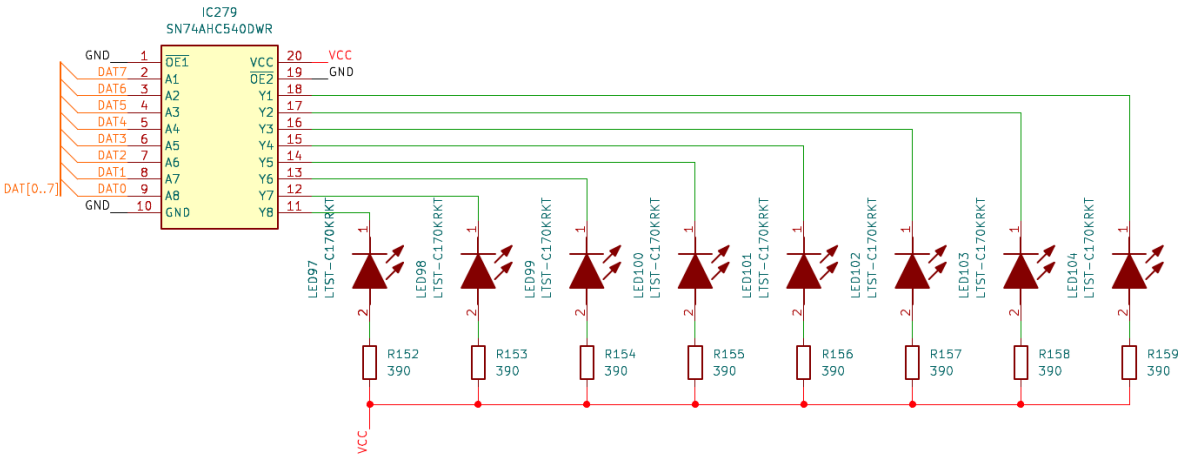


Figure 65: DAT Bus LED indication circuit

In order to be able to connect the LEDs to the Bus an inverter such as the SN74AHC540 is needed to flip the logic level of the input signal that the LED turns on when, for example, DAT0 is HIGH. This works because the voltage difference, when DAT0 is HIGH, is the voltage the LED needs to shine. To Limit the current to 8mA, 390Ω resistors are used because the inverter IC cannot handle more than 75mA of continuous current through V_{CC} or GND.

4.5 Printed circuit board

For a better debuggability the 8Bit-Computer was split into 4 PCBs which are connected via 2.54mm Pin Header and Pin Sockets. Furthermore, each PCB has its own 20V to 5V DC-DC Converter and because of this its own independent 5V source.

4.5.1 PCB Board Setup

The PCBs were designed with four Layers. The Top and Bottom Layers are used for Signal wires, while the first middle Layer is a Ground region and the second middle Layer is a 5V V_{CC} region. In addition, the unused areas on the Top and Bottom Layers were filled with Ground regions to avoid any electromagnetic interference and to provide shielding against any external interferences.

4.5.2 Drawing of the Printed circuit board

While drawing the PCB, the signal Track Width is 0.15mm and spacing between tracks is 0.1mm. For V_{CC} and GND the Track Width is 0.6mm and spacing is 0.1mm. For Via Size 0.5mm and Via Hole 0.3mm were taken.

4.5.2.1 *Power distribution and conversion*

To distribute the V_{BUS} 20V of the USB C PD Board to the DC-DC Converters on each PCB, a calculation of the needed wire thickness of the V_{BUS} was calculated with the built in Calculator Tools in KiCad.

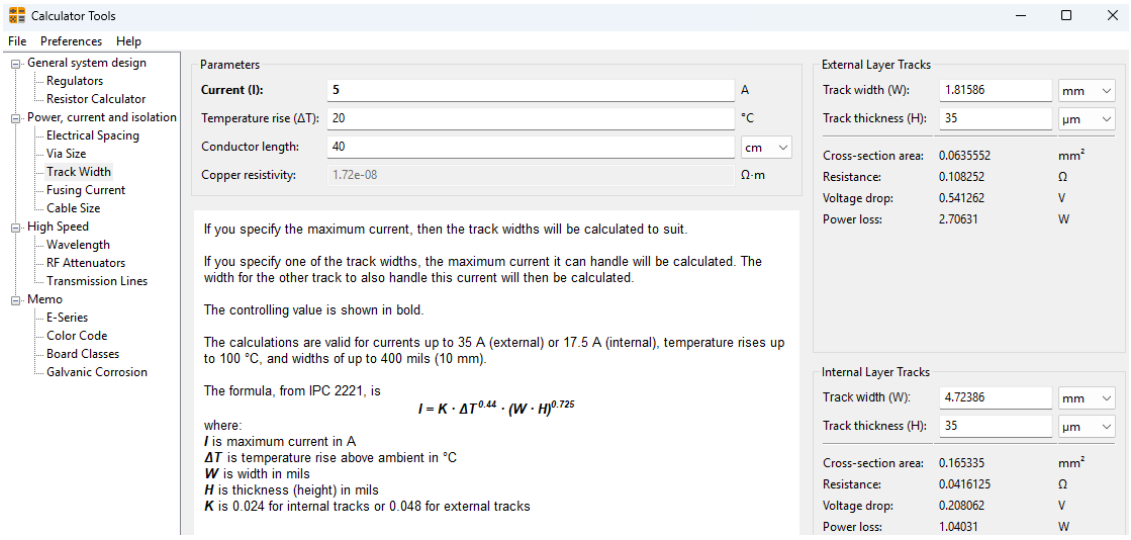


Figure 66: PCB V_{Bus} wire thickness calculation

The 5A that the PD Board provides, the maximum Temperature rise with 20°C and the length of 40cm were set and track widths with 4.7mm for internal Layers and 1.8mm for external Layers (Top and Bottom Layers) were calculated. To have little headroom the widths were rounded up to 6mm and 3mm for V_{Bus}. Because of the lower Power loss and Voltage drop compared to the external Layer, the V_{Bus} was routed in the inner V_{CC} Layer from the Pin Headers of one Board to the other Headers of the other Boards. In the following figure the traced DC-DC Converter circuit can be seen.

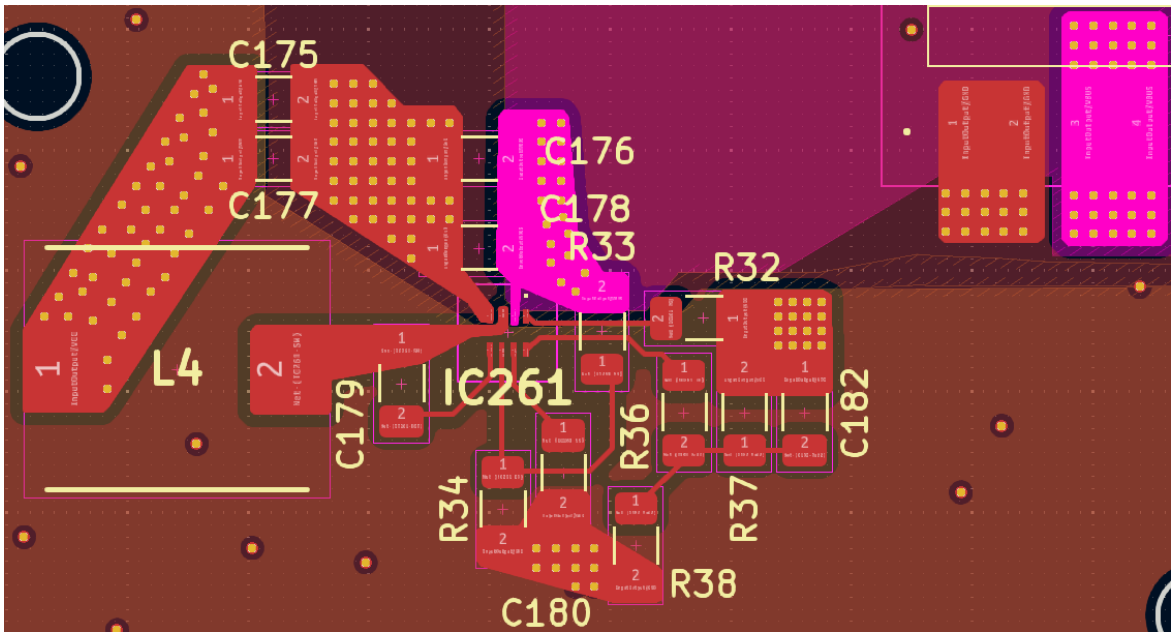


Figure 67: DC-DC Converter PCB circuit

4.5.2.2 PCB example Memory Module

On the Memory Module the Flash Memories with its Programming interfaces and the RAM were placed.

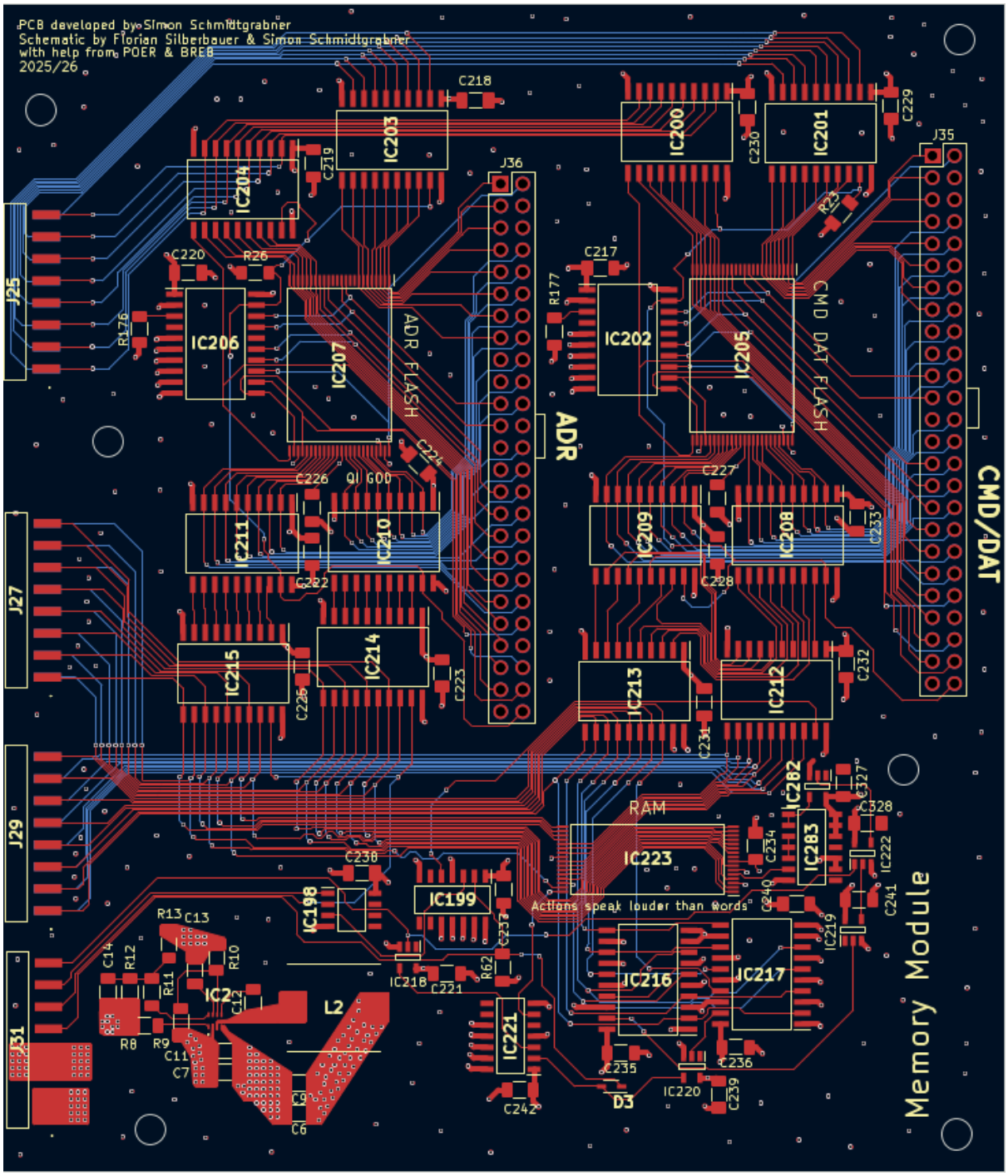


Figure 68: Memory Module PCB

4.6 Assembly

4.6.1 SMD Assembly

The SMD Assembly Process of a PCB consists of three steps:

1. Firstly, the stencil is placed on the PCB then the solder paste is put on.
2. Secondly, the components are placed on the solder pads which are covered in solder paste.
3. Lastly, the PCB with its components is put into a reflow oven to heat up the paste to its melting point and then cooled down for a solder connection to establish.

4.6.1.1.1 Applying Solder paste

For the assembly process stencils for every PCB were used to apply solder paste. For example, the following picture shows the application of the solder paste on the Control Module.

Figure 69: Solder paste application

4.6.1.1.2 Reflow oven

The profile used for the reflow oven is as follows:

1. Preheat with 150°C for 170s
2. Reflow with 240° for 115s
3. Cool time 100s

4.6.2 THT Assembly

The components that are through-hole like the switches and the Flash programming pins were soldered with a soldering iron.

4.7 Test of individual circuit boards

The tests of every individual board consisted of two parts. The first test was to validate that there a no short circuits on the power supply connection and the second test to check that there is no more than one command decoder LED light up when selecting a specific command.

4.7.1 Short circuit testing

To test for short circuits a laboratory power supply was used. The power supply was set to 5V and the current limitation to 1mA. The power supply was then connected to the board via a V_{CC} pin. In the next step the current limitation is slowly raised while monitoring the voltage of the power supply and the temperature of every IC with a thermal camera. If any of the ICs would be broken the thermal camera would capture it.

The same procedure was repeated to test each Dc-Dc converter on every board. For this the PSU was set to 20V and connected to the V_{BUS} pin on the PCB.

4.7.2 Command decoder testing

In order to test the functionality of every command a tester was built with slide switches to enter every command on every PCB. To enter the commands the red wire was connected to 5V, the black wire to GND and the pin socket to the CMD pin header on every PCB.

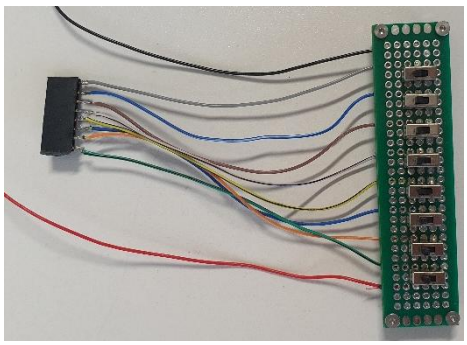


Figure 70: Command decoder tester

4.7.3 Found and corrected errors

The errors found and corrected are listed in the following:

4.7.3.1 Errors I/O Module

Dat_{dis} on Diode D28, D29, D31 were connected wrongly between CMD decoder and Inverter and not after the inverter

CMD decoder IC243 and IC237 had comparison pins connected to the wrong voltage potential.

The delayed signal from the rising edge detector circuit was changed from the DS1100-150+ to a 3 times 2 input OR gate connected in serial.

4.7.3.2 Errors Control Module

The inverter IC323 was added to the CLKd wire for it to be inverted to $\overline{\mathrm{CLKd}}$.

A debounce capacitor with 100nF was added in parallel to the signal wire and GND of the Button Clock.

In the Data to Address Register one D-latch IC and the second half of the D-latch that was already in use were added for the address to be delayed for two whole clock cycles (see chapter 4.4.2.4).

4.7.3.3 Errors ALU

CMD decoder IC39 and IC40 had comparison pins connected to the wrong voltage potential.

A DAT_{dis} diode was forgotten for the shift cmd in the barrel shifter register now added with the name D15.

4.7.3.4 Errors Memory Module

The additional circuitry for writing on the RAM was added (see chapter 4.4.2.3).

The output of the DAT_{dis} D-latch IC199 was changed from $\overline{1Q}$ to 1Q.

The second D-latch on the 2 times D-latch SN74AHC74DR IC199 for ADR_{dis} was put into unused state.

The RP# pin from both of the Flash memories was pulled up to 5V via a bus driver when not in programming mode. (IC202 and IC206)

The pull-down resistors for the ProgModeADR / ProgModeCD wires were reduced to one 4,7kΩ resistor each.

4.7.4 Found and not corrected errors

The only error that was not corrected up to the date of publishing the diploma thesis is currently a unknown problem with the Flag Register (see chapter 3.4.5.5). This is not a major error since the function of the Flag Register can be realised with some effort in software.

4.8 Test of the complete circuit board

To test the complete circuit board every individual board were connected together and a test programme was loaded on the Flash memories to test every functionality.

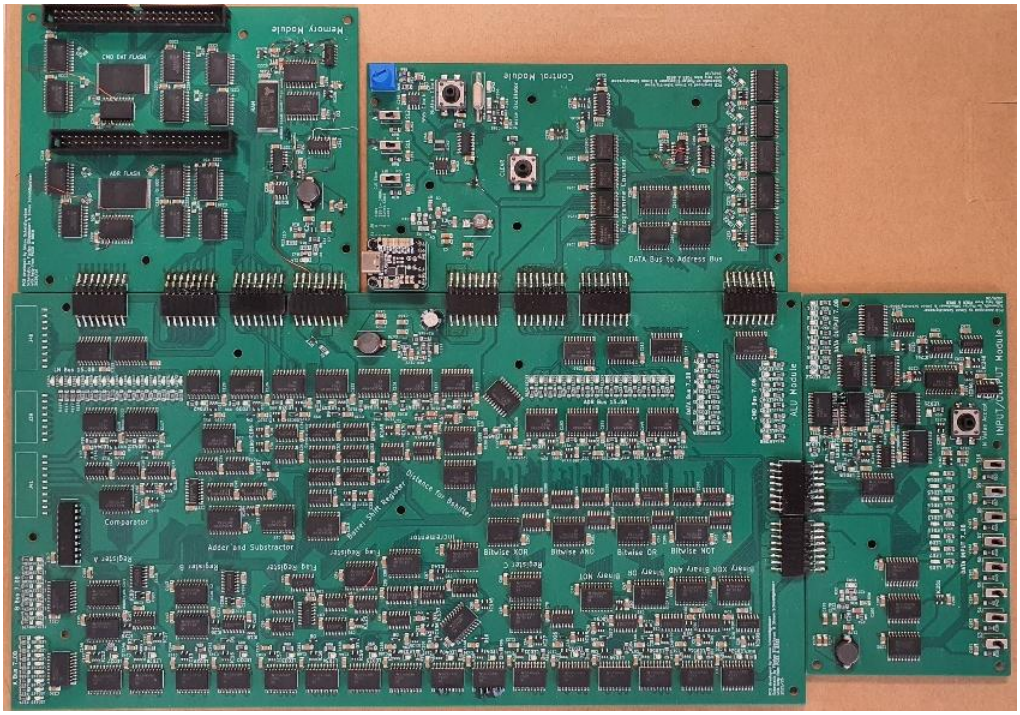


Figure 71: Complete 8Bit-Computer PCB

4.9 Enclosure for the circuit boards

In order to protect the Computer from electrostatic discharge because most of the ICs used are CMOS based and ESD safe to only <2000V an enclosure is needed.

Firstly, standoffs will be inserted into a wooden board which serves as the base plate. Secondly, the PCB is secured with a standoff as a screw. Thirdly, a sheet of acrylic or plexiglass is secured with a screw into the standoff.

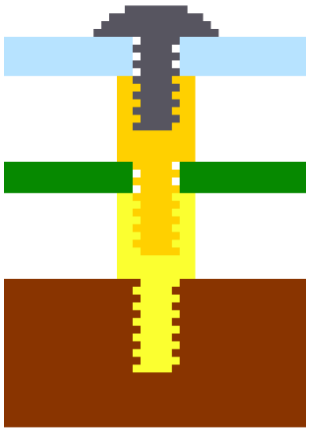


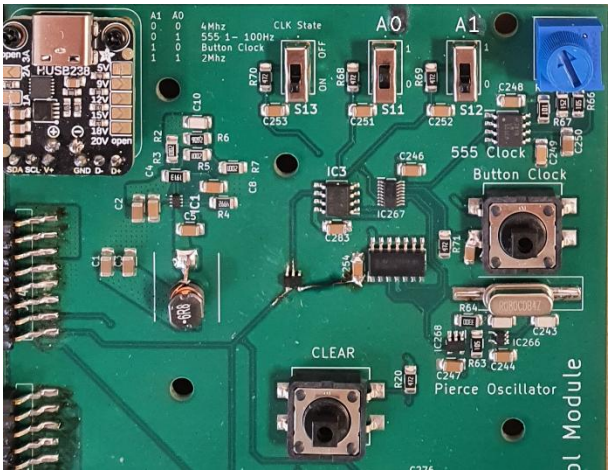
Figure 72: PCB Enclosure

5 Operating instructions – Simon Schmidtgrabner

In the following chapter is explained how to use and program the 8Bit-Computer

5.1 Selection of the clock generators

With the switches on the Control Module the clock can be selected according to the table (Table 24: Clock Multiplexer Truth Table) in chapter 4.4.2.5.1 or the table printed on the PCB.



In addition, the Clock can be completely deactivated by sliding the CLK State switch to OFF.

5.2 Programming the address and command/data flash memories

To program the Flash memories first install the software for the Xgecu T76 programmer (see chapter 4.2)and driver with the name Xgpro T76 found on the official website of Xegcu. Then start up the Xgpro T76 programme, on the first start up the following GUI will open.

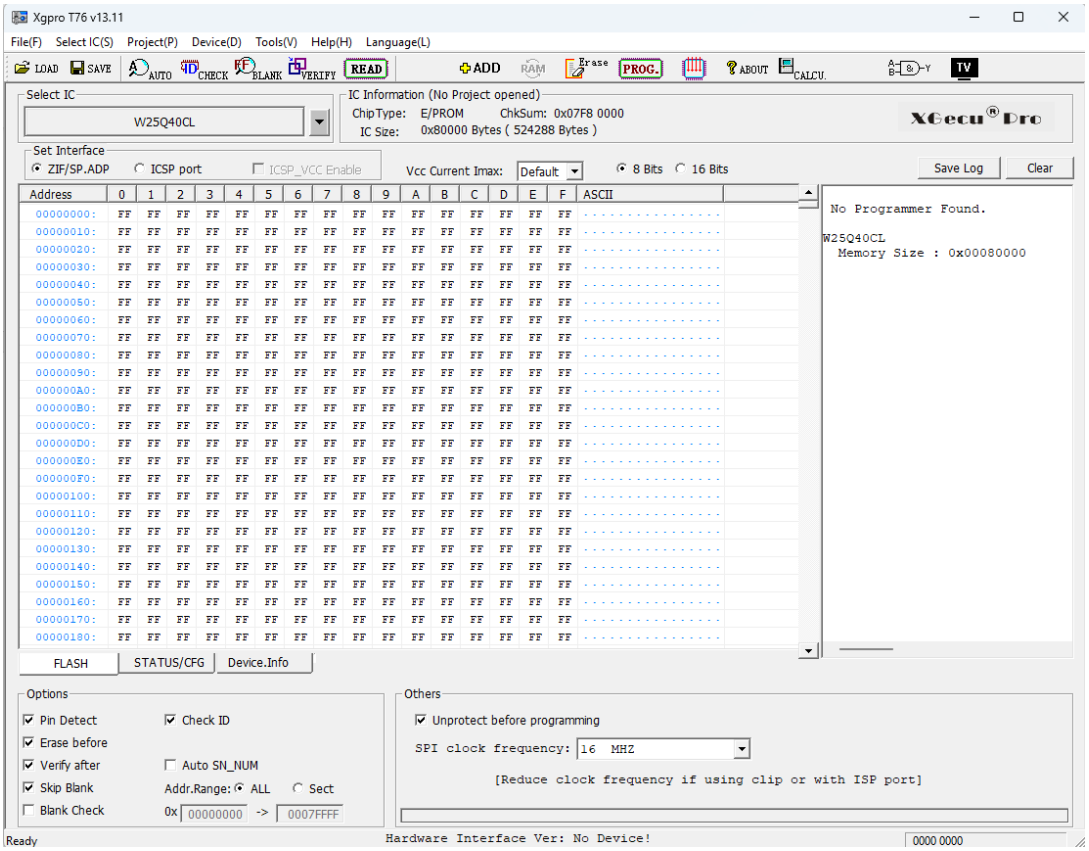


Figure 73: Xgpro T76 programme Start up

Click on the “Select IC” Button and search with “Full” and “Type ALL” ticked on for “M29F800FT” pick the one under ST with the Device name “M29F800FT @TSOP48” and click the “Select” button.

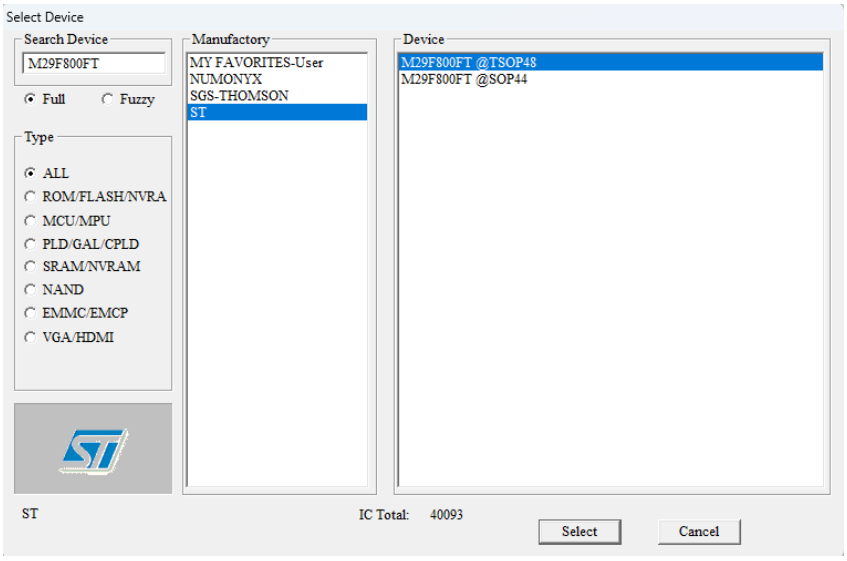


Figure 74: Xgpro T76 Device select

Deselect “Pin Detect” and “Check ID” as shown in the Figure.

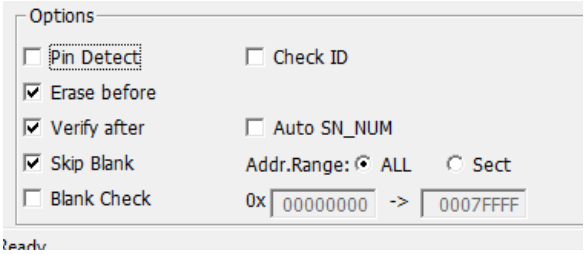


Figure 75: Xgpro T76 programme Options window

To load compiled programmes (see 6.3) click on the button “LOAD” and a new window will open. Select here the compiled programme via the “Browse” button and change the “Clear Buffer when loading the file” to “Clear buffer with 0x00”.

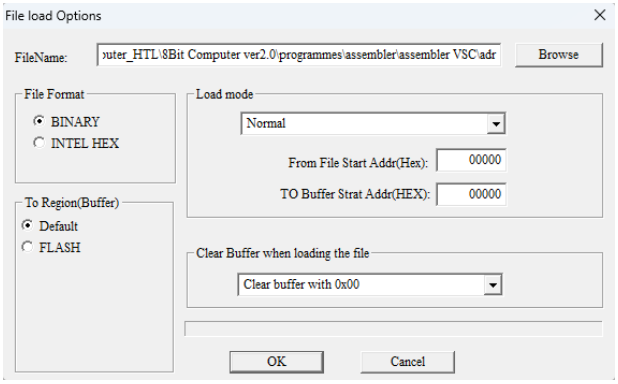


Figure 76: Xgpro T76 File load Options window

Now plug the Xgecu T76 programmer in with USB and turn on the Programmer via the Power button near the USB C port then the Xgpro T76 programme will show on the right side in the LOG that the Programmer is Connected. With the shown Data depending on the compiled programme that was loaded in. Here the ADR was loaded in.

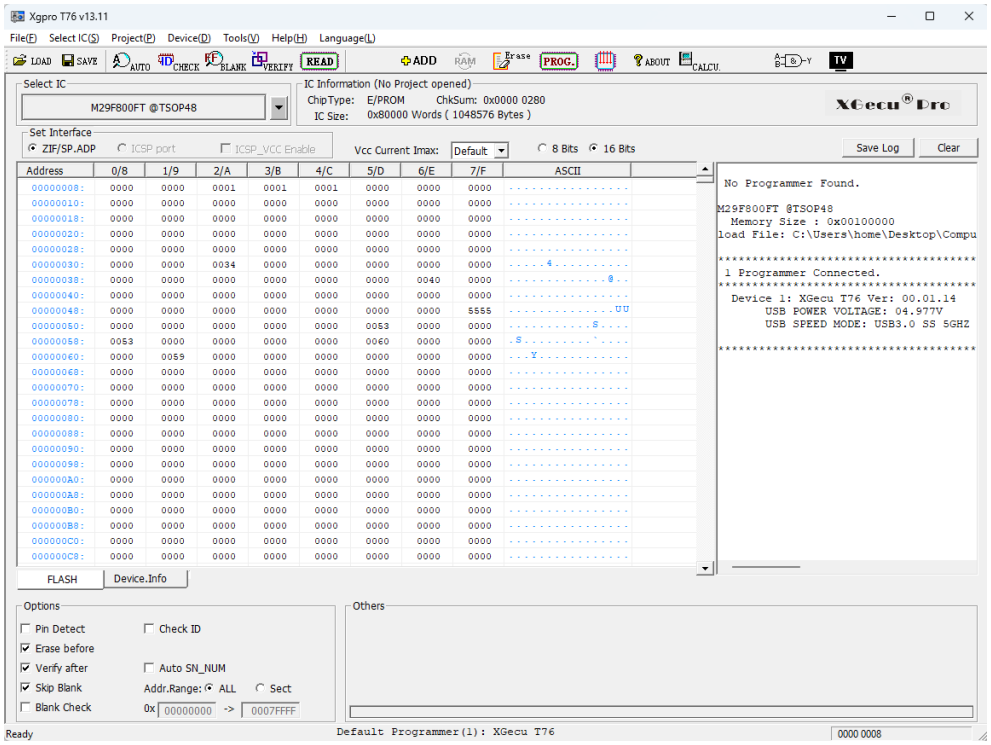


Figure 77: Xgpro T76 programme pre-Flash setup

Be sure to plug the cable Converter in the correct way as shown in the figure on the right. The Row count on the side of the PCB must be side as the lever of the Controller seen in the figure.

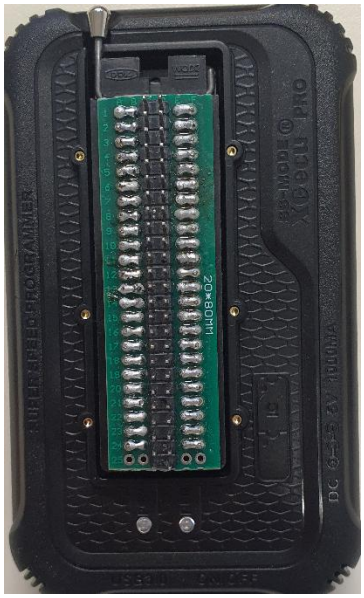


Figure 78: T76 Programmer and Cable converter

Now the cable can be plugged in to the converter on the programmer. The keyed side of the cable must always be on the side where “20*80MM” is printed on the converter PCB as shown in the Figure 80 below.

Figure 80: Front view of the Cable plugged into the Converter

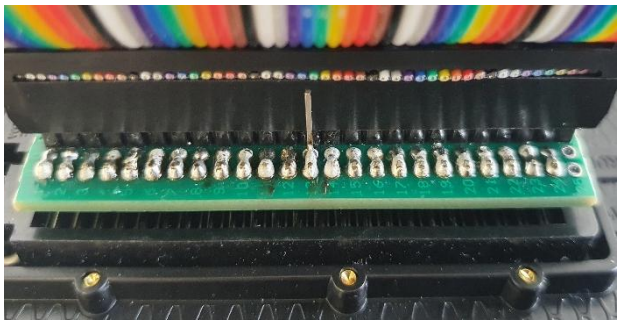


Figure 79: Back view of the Cable plugged into the Converter

Before plugging the programming cable into one of the Flash program interface pin headers select the 555-Timer clock and hold the CLEAR Button on the Control Module while deactivating the Clock signal via sliding the CLK State switch to OFF. This is done to reset any data that is on the buses.

With everything set up for programming, plug in the other side of the cable into one of the two programming interfaces shown in the Figure below. For Address connect it to ADR and for Command and Data connect it to CMD/DAT.

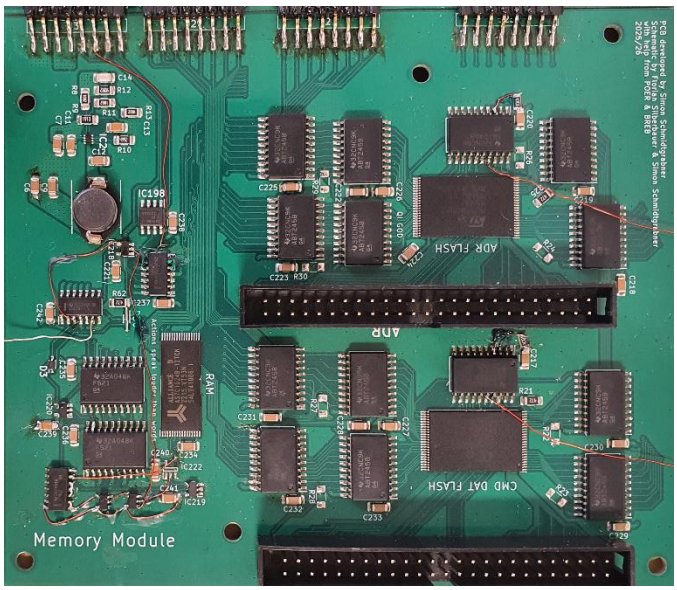


Figure 81: Memory Module

5.3 Input and Output of the data values

To input values, use the slide switches on the INPUT/ OUTPUT Module and confirm them via pressing the IN Value Accept button. The input values will also be shown with the LEDs located above the switches.

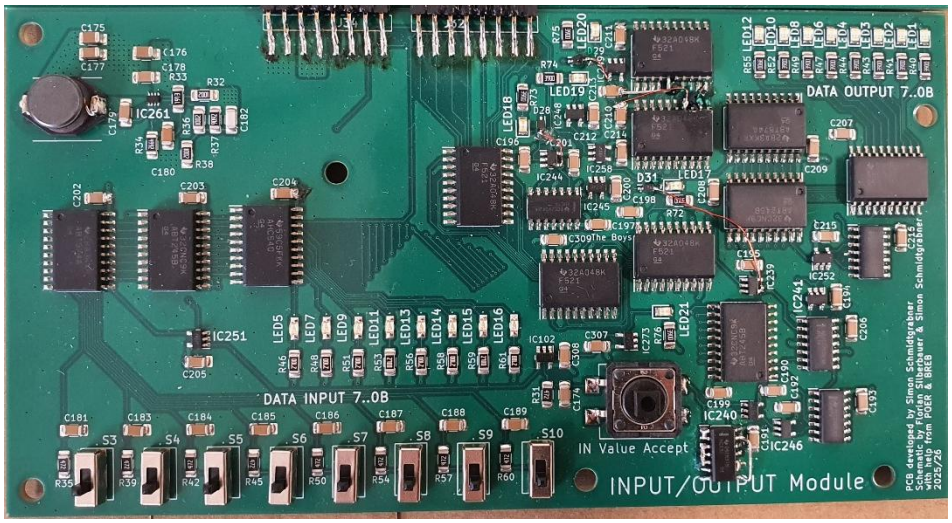


Figure 82: INPUT/OUTPUT Module

The output data can be seen on the top right corner shown via 8 LEDs in Binary

6 Assembler – Florian Silberbauer

The goal of this project was to make a computer, which could be programmed by the user. In order to simplify the programming, a custom assembler was made.

6.1 Difference between an Assembler and a Compiler

For it to be understandable why an assembler was chosen, opposed to a compiler, which is the usual method of creating runnable code, it is important to first know what the difference of these two options is.

6.1.1 Compilers

They are used for compiling higher level languages, which are based on different lower-level languages. C, for example, is based on Assembly, while Python is based on C. Almost all modern programming languages are in some way based on a lower-level language.

Since they are used for higher-level languages, they need to translate them to the lower-level ones, on which they are based. While doing so, they look for patterns in the code, which can be simplified and optimised. They usually translate their language down to assembly, which is then handled by an assembler.

Compilers also handle memory management, along with other ease-of-use features.

6.1.2 Assemblers

Assemblers are used in order to translate Assembly into machine language, which the computer can understand. Since compilers only translate up to assembly, assemblers are also required when working with compilers.

Since assembly is very close to human-readable machine language, assemblers just translate commands written in assembly to their binary equivalent. Things like memory management need to be handled by the user.

6.2 Custom Assembly

Since the computer has a custom architecture, the code that needs to be written for it, is different from any existing versions of assembly, like 6502 assembly, upon which all the modern CPUs are based on.

There are multiple types of commands defined in this custom assembly language:

- Commands used for loading and storing
- Commands meant to perform operations on data
- Commands for jumping

There is also another command called “nop” (No Operation), which does nothing for one cycle.

Commands for storing will take the values on the bus at that moment and store them to a register or to RAM. When storing to RAM, an address is required. When commands need to load values, they will do so during the next cycle, for other commands to be able to store said value. Commands used for performing operations on data will do so and then output the result to the bus like commands meant for loading.

Another kind is meant for jumping. They are used for flow control in structures such as if statements. Jumping works simply by changing the value of the programme counter. Essentially, this “jumps” through the list of commands stored in ROM.

6.2.1 Lists of Commands

This chapter is an introduction into writing and understanding programmes for the computer, as all the commands needed for the computer are listed and explained. The arguments for the commands need to be present as the command begins to execute, while the outputs are on the bus one cycle later, in order to give commands the ability to use the data. A section of example code using the commands explained here can be found further down in chapter 6.3.4, Example Assembly Code.

6.2.1.1 General Commands

These commands do not necessarily fit any of the other categories, are however still important in the operation of the computer.

Core			
Command	**Arguments**	**Output to**	**Description**
nop	-	-	No Operation
jmp	Address	-	Jump
cjp	Address, Data	-	Jump if

Table 25: Core Commands

The output of the Bus Transfer is delayed by two cycles, in order to allow for data to be loaded to the bus in the following cycle, after which the address will be written to the bus, which can be used for storing to locations in RAM, as specified by the programme.

Bus Transfer			
Command	**Arguments**	**Output to**	**Description**
stx	Data	-	Store to interface reg 1
sty	Data	-	Store to interface reg 2
ldz	-	Address Bus (by 2 cycles)	Load interface regs

Table 26: Bus Transfer Commands

RAM			
Command	**Arguments**	**Output to**	**Description**
sto	Address, Data	-	Store to RAM
lod	Address	Data Bus	Load from RAM

Table 27: RAM Commands

6.2.1.2 Commands within the ALU

These commands are all located in the ALU, and most of them are connected to registers. The way to interact with these commands is to store values to the A/B/C registers. There are two more Registers in the ALU, which both have control functionality. The flag register is meant to store the state in which the last addition or subtraction exited. The distance register is used to control the behaviour of the shift register.

Adder			
Command	**Arguments**	**Output to**	**Description**
add	Reg A/B	Data Bus	Add
adc	Reg A/B	Data Bus	Add with carry
sub	Reg A/B	Data Bus	Subtract

Table 28: Adder Commands

Incrementor			
Command	**Arguments**	**Output to**	**Description**
inc	Reg C	Data Bus	Increment
dec	Reg C	Data Bus	Decrement

Table 29: Incrementor Commands

Logic			
Command	**Arguments**	**Output to**	**Description**
not	Reg A	Data Bus	Binary Not
orr	Reg A/B	Data Bus	Binary Or
and	Reg A/B	Data Bus	Binary And
xor	Reg A/B	Data Bus	Binary Xor
bnt	Reg A/B	Data Bus	Bitwise Not
bor	Reg A/B	Data Bus	Bitwise Or
ban	Reg A/B	Data Bus	Bitwise And
bxr	Reg A/B	Data Bus	Bitwise Xor

Table 30: Logic Commands

Compare			
Command	**Arguments**	**Output to**	**Description**
cgt	Reg A/B	Data Bus	Compare Greater than
ceq	Reg A/B	Data Bus	Compare Equal
clt	Reg A/B	Data Bus	Compare Smaller than

Table 31: Comparator Commands

A/B/C Registers			
Command	**Arguments**	**Output to**	**Description**
sta	Data	-	Store A
lda	-	Data Bus	Load A
stb	Data	-	Store B
ldb	-	Data Bus	Load B
stc	Data	-	Store C
ldc	-	Data Bus	Load C

Table 32: A/B/C Register Commands

Flag Register			
Command	**Arguments**	**Output to**	**Description**
stf	Data	-	Store Flags
lcr	-	Data Bus	Load Carry
ldo	-	Data Bus	Load Overflow
ldf	-	Data Bus	Load Flags

Table 33: Flag Register Commands

Shift Register			
Command	**Arguments**	**Output to**	**Description**
std	Data	-	Store Distance
ldd	-	Data Bus	Load Distance
sft	Reg C	Data Bus	Shift
sfc	Reg C	Data Bus	Shift Circular

Table 34: Shift Register Commands

6.2.1.3 Input and Output Commands

The ldu command loads the unread bit, which is set whenever a new input is stored. It is however reset after reading the value. This needs to be taken into account when writing programmes which are meant to interact with the user.

Inputs			
Command	**Arguments**	**Output to**	**Description**
sti	Data	-	Store Inputs
ldi	-	Data Bus	Load Inputs
ldu	-	Data Bus	Load Unread

Table 35: Input Commands

Outputs			
Command	**Arguments**	**Output to**	**Description**
sop	Data	-	Store Outputs
lop	-	Data Bus	Load Outputs

Table 36: Output Commands

6.2.1.4 Unused Commands

These commands are implemented in the assembler, but since they were not implemented in the physical version, they can only be used in the simulation.

Screen			
Command	**Arguments**	**Output to**	**Description**
str	Address, Data	-	Store Row
ldr	Address	Data Bus	Load Row

Table 37: Screen Commands

Controller			
Command	**Arguments**	**Output to**	**Description**
lin	-	Data Bus	Load Inputs

Table 38: Controller Commands

6.3 Assembler Programme

For realising a custom version of Assembly, a custom assembler was needed. This assembler was given the minimum of features with which it would still be usable.

6.3.1 Usage

The assembler programme is an executable, which assembles the code to its binary version when opened. It does not open a window.

Code is written to a text file, located in the same folder as the programme, which needs to be named “code.txt”. When the executable is run, it will replace any wrong line numbers and output errors to “errors.txt”. The output of the programme are two binary files, one called “cmd”, with commands and data, and another one called “adr”, with the addresses.

6.3.2 Features

6.3.2.1 Comments

Comments are a necessity for any programming language, as it is possible to comment out parts of the code, in order to test the function of a programme. They are also indispensable for explaining parts of a programme.

6.3.2.2 Line Numbers

They show the exact position the command will be stored at in ROM. This knowledge is important for jumping and simply knowing the structure of the programme.

When assembling the programme, the assembler rewrites the line numbers at the beginning of every line, if they have a semicolon on their first position. If a command does not have a semicolon at the beginning, it is not recognized as such.

6.3.2.3 Labels

Labels are used for jumping, so that the user does not have to update the address of every jump command once a new line gets added at the top of the programme.

They are defined at the beginning of the code, and their subsequent use defines their position. When put next to a “nop”, they will remember the address, and when put next to a jump command, they will put that address at that location.

Labels can be jumped to from multiple locations, but the location to which they point can only be defined once.

6.3.3 Source Code

The code was written in C++, making heavy use of classes.

6.3.3.1 Libraries

The cstdint, iostream, and fstream libraries are required for interaction with files. The bits/stdc++.h library was used for converting from integers to hexadecimal strings for readabilities' sake. The rest are used throughout the programme.

```
2   #include <cstdint>
3   #include <iostream>
4   #include <fstream>
5   #include <string>
6   #include <list>
7   #include <bits/stdc++.h>
```

Figure 83: Libraries used for the Assembler

6.3.3.2 Classes

6.3.3.2.1 Enumerators

```
12  enum Requires{
13      Nothing,
14      Data,
15      Address,
16      Both
17  };
18
19  enum Disables{
20      None,
21      DataBus,
22      AddressBus
23  };
24
25  enum Duration{
26      oneCycle,
27      twoCycles
28  };
```

Figure 84: Enumerators in the Assembler

These enumerators are meant to help with the constructors of the following classes.

6.3.3.2.2 Command Class

```
class Cmd {
public:

    void Init(string nameIn, uint8_t valIn, Requires reqIn, Duration durIn, Disables disIn) {
        name = nameIn;
        val = valIn;
        req = reqIn;
        dis = disIn;
        dur = durIn;
    }

    string Name() {
        return name;
    }

    uint8_t Put() {
        return val;
    }

    uint8_t Req(Requires reqIn) {
        return (req == reqIn);
    }

    Disables Dis() {
        return dis;
    }

    Duration Dur() {
        return dur;
    }

private:
    string name;
    uint8_t val;
    Requires req;
    Disables dis;
    Duration dur;
};
```

Figure 85: The Command Class

The command class simply stores the name, the opcode, and some other information to help with error detection. The methods included are only getters and setters, so that the variables cannot directly be interacted with by the programme.

6.3.3.2.3 Label Class

```
class Label {
public:
    void SetName(string nameIn) {
        name = nameIn;
    }

    void AddPosition(uint16_t lineIn) {
        line.push_back(lineIn);
    }

    void SetDestination(uint16_t toIn) {
        to = toIn;
    }

    string GetName() {
        return name;
    }

    list<uint16_t> GetPositions() {
        return line;
    }

    uint16_t GetDestination() {
        return to;
    }

private:
    string name = "undef.";
    list<uint16_t> line;
    uint16_t to = 0;
};
```

Figure 86: The Label Class

This class is meant to store the name and locations of labels. The line list is used for storing the locations at which a label gets called. Here, just as in the command class, all the defined methods are getters and setters.

6.3.3.3 Functions

These functions are used for simplifying the code.

6.3.3.3.1 Create List

Create List groups together all the command class definitions and adds them to the global commands list.

```
359  void CreateList() {
360
361      Cmd cmd;
362
363      // --- CORE ---
364      cmd.Init("nop", 0x00, Requires::Nothing, Duration::oneCycle, Disables::None); Commands.push_back(cmd); // No Operation
365      cmd.Init("jmp", 0x01, Requires::Address, Duration::oneCycle, Disables::None); Commands.push_back(cmd); // Jump
366      cmd.Init("cjp", 0x02, Requires::Both,    Duration::oneCycle, Disables::None); Commands.push_back(cmd); // Jump if
```

Figure 87: The Create List Function

6.3.3.3.2 To Hex

```
442  string ToHex(uint16_t num, uint8_t len) {
443
444      string out;
445
446      for(int i; i < len; i++) {
447          out = out + "0";
448      }
449
450      try {
451          stringstream s;
452          s << hex << (int)num;
453
454          uint8_t strLen = s.str().length();
455
456          out = out.insert((len-strLen), s.str());
457          out = out.substr(0, len);
458
459          transform(out.begin(), out.end(), out.begin(),::toupper);
460      }
461      catch(exception e) {}
462
463      return out;
464  }
```

This function takes in an integer and returns the hexadecimal version as a string, with the specified number of symbols.

Figure 88: The To Hex Function

6.3.3.4 File Creation

Here, all the important files are opened. The command and data files are created as binary ones, since additional formatting symbols are added to text files.

```
107  int main() {
108
109      CreateList();
110
111      /* Open the code to be read from */
112      ifstream codeFile("code.txt");
113
114      /* temporary file for rewriting code */
115      ofstream tmpCode("tmp.txt");
116
117      /* Create the hex files for programming the computer */
118      ofstream cmdFile("cmd", ios::binary); //cmd and dat in one file, since they are combined in one chip
119      ofstream adrFile("adr", ios::binary);
120
121      /* Create a txt for logging errors */
122      ofstream errorLog("errors.txt");
123
124
125
126      string newLine;
127      uint8_t check;
128
129      uint16_t currentLine = 0;
130
131      Disables prevBusAccess = None;
132      Disables prevToLastBusAccess = None;
133
134      uint16_t prevAdr = 0;
135      Duration prevAdrDuration = oneCycle;
```

Figure 89: File Creation in the Assembler

6.3.3.5 Structure

The programme is made up of two while loops within the main function. The first is meant to parse the code first and find the definitions of labels along with all their positions within the code, while the second is meant to fill the binary files with the correct values, where the labels, found earlier, are also used for defining addresses next to jump commands.

6.3.3.5.1 Line Cleanup

In order to be able to read the programme, the first step of assembling is to ignore any empty or commented lines. Next, everything up to the first semicolon is removed, since it is not required for the assembler. At line 233, the line numbers are rewritten to the temporary file, to which the code is copied. After the assembler is done, it will copy the text from that file to the main code file and then delete the temporary one.

```
218         /* skips empty lines, ones with comments, or label definitions */
219         if(newLine.empty() or (newLine.find("//") == 0 ) or (newLine.find("label:") == 0)) {
220             tmpCode << newLine << endl;
221             continue;
222         }
223 
224         /* beginn reading the code from the first semicolon */
225         uint8_t pos = newLine.find(";") + 1;
226         newLine = newLine.substr(pos);
227 
228         /* cut out any spaces */
229         pos = newLine.find_first_not_of(' ');
230         newLine = newLine.substr(pos);
231 
232         /* write new line numbers at the beginning of the lines */
233         tmpCode << ToHex(currentLine, 4) + "; " + newLine << endl;
234 
235         /* in order to avoid an empty string later */
236         newLine = newLine.append("                ");
```

Figure 90: Code for cleaning Lines of Assembly Code

6.3.3.5.2 Filling Binary Files

This part of the programme only gets activated if a command name is found on the current line. This detection is done within a foreach loop of the commands list. This loop happens for every line of the assembly code. The opcode of the found command is placed in the cmd file without question. The data is first set to zero, and is only changed to something else, if the command needs a value, and one is provided. Otherwise, an error gets placed in “errors.txt”.

```
240         for(Cmd cmd : Commands) {
241 
242             if(cmd.Name() == newLine.substr(0, newLine.find(' '))) {
243 
244                 /* write cmd value to the file */
245                 uint8_t b = cmd.Put();
246                 cmdFile.write(reinterpret_cast<char*>(&b), 1);
247 
248                 /* cmd XX ---- */
249                 uint8_t val = 0x00;
250 
251                 /*
252                  * only if the command needs data, and no previous command
253                  * is outputting data to the bus
254                  */
255                 if((cmd.Req(Requires::Data) or cmd.Req(Requires::Both)) and
256                 !(prevBusAccess == Disables::DataBus)) {
257 
258                     /* try to write 8 data bits to the cmd file */
259                     try{
260                         newLine = newLine.substr(newLine.find(' ') + 1);
261                         string s = newLine.substr(0, 2);
262                         val = stoul(s, nullptr, 16);
263                     }
264                     catch(exception e) {
265                         errorLog << "unknown data value at line: " << ToHex(currentLine, 4) << endl;
266                     }
267                 }
268                 cmdFile.write(reinterpret_cast<char*>(&val), 1);
```

Figure 91: Code Partition for Filling Binary Files with Commands and Data

Writing addresses to the binary files is a bit more complicated. As with the data and command file, the address value is set to zero.

First, nested foreach loops get called, checking through the list of positions where labels are called. If the current line matches to one of the lines, the address value is changed to the value stored within the label.

If no label was found, and a command requires an address, the assembler will place the address written in the assembly code into the binary file.

If a command needs a value on the bus for more than one cycle, it will set the address to be the same value as in the previous cycle.

```
270             /* cmd -- XXXX */
271             uint16_t adr = 0x0000;
272             bool labelDetected = false;
273
274             /* if there's a label at this position, it will write the destination to adr */
275             for(Label label : Labels) {
276
277                 for(uint16_t i : label.GetPositions()) {
278
279                     if(i == currentLine) {
280                         adr = label.GetDestination();
281                         labelDetected = true;
282
283                         errorLog << "replaced address at line " << ToHex(currentLine, 4);
284                         errorLog << " with: " << ToHex(label.GetDestination(), 4);
285                         errorLog << " > " << label.GetName() << endl;
286                     }
287                 }
288             }
289
290             /*
291              * if the command needs an address, and no previous command
292              * is outputting an address to the bus
293              */
294             if((cmd.Req(Requires::Address) or cmd.Req(Requires::Both)) and
295             !(prevToLastBusAccess == Disables::AddressBus) and !labelDetected) {
296
297                 /* try to write 16bits to the adr file */
298                 try{
299                     newLine = newLine.substr(newLine.find(' ') + 1);
300                     string s = newLine.substr(0, 4);
301                     adr = stoul(s, nullptr, 16);
302                     prevAdr = adr;
303                 }
304                 catch(exception e) {
305                     errorLog << "unknown address value at line: " << ToHex(currentLine, 4) << endl;
306                 }
307             }
308
309             /* in order to keep addresses on for one cycle longer */
310             if(prevAdrDuration == twoCycles) {
311                 adr = prevAdr;
312             }
313             adrFile.write(reinterpret_cast<char*>(&adr), 2);
```

Figure 92: Code Partition for Filling Binary Files with Addresses

6.3.4 Example Assembly Code

```
label: start
label: input

//------ setup ------------------------------------------------
0000; nop
0001; sta 00
0002; stb 00
0003; stc 00
0004; std 00
0005; sop 00
0006; sti 00
0007; stf 00
0008; stx 00
0009; sty 00
000A; nop start

// ------ wait for input --------------------------------------
000B; nop input
000C; ldu
000D; sta
000E; not
000F; cjp input // /\ loop back for inputs /\

// ------ store input value -----------------------------------
// store values to RAM
0010; ldc
0011; stx
0012; ldz
0013; ldi
0014; sto

// increment input counter
0015; inc
0016; stc
0017; jmp start
```

Figure 93: Example Code written in Assembly

This Code snippet is meant to store values from the input register to RAM. The address at which the values are stored is incremented starting from zero.

First, all the labels needed for the programme are specified with the “label:” call. The code in the setup section stores zeroes to all the registers located in the computer. The “start” label specified at line 000A is meant for the programme to return to when it is finished, for it to repeat indefinitely.

Next, in the “wait for input” section, the code continually loads the unread register, which is only high when the user presses the input button. The value gets stored to the A register, where it is inverted. Then, the conditional jump command gets called at line 000F. This means

it will jump whenever the value from the unread register is zero. When it jumps, it returns to the beginning of the section, for it to keep checking the unread register. When the unread register has a high state stored, the conditional jump does not get activated, and the programme is allowed to continue.

The “store input value” section stores the input value to RAM. This is achieved by loading the value from the C register and storing it to the first interface register, which is loaded to the address bus. Then the input register is loaded to the data bus, and with the data and address on the bus, the store command is called. After this, the C register gets incremented to the next position. At the end, the programme jumps back to the beginning with the “start” label.

6.3.5 Problems Encountered during Programming

Common Problems like writing to temporary variables, wrong typecasts, and forgetting to handle exceptions showed up. One notable Problem, which also cost a considerable amount of time, was forgetting to set the filetype of the binary output files to binary, since text files use additional formatting. When, for example, a newline symbol is put in a text file, the programme will check if there is a line end symbol before it, and if there is not, it will place one there. This shifts all the values coming after it over by 8 bits, making the code behave unpredictably when running it on the computer.

7 Appendixes

7.1 Timetable

7.1.1 Florian Silberbauer

	Task	Hours Spent
2022 - 2023	V1 - not counted here	-
20/06/2023	Add files via upload	10
03/02/2024	Think up ideas on the design	4
07/02/2024	Consider adding a Multiplication unit	6
09/03/2024	Look through 74 series chips + think through an example programme along with the timing	8
30/03/2024	Think about possible problems + possibilities of adding Interrupts	7
13/04/2024	Write an addition programme without overflow on paper to think through	8
29/05/2024	Sorting	3
18/04/2025	added buffer to the list + made 8bit parallel adder	6
29/04/2025	fixed bitwise and	8
22/05/2025	translation from 74 series designs to logical designs	7
12/06/2025	created the adder file	6
21/06/2025	added commands to the list + redid previous order	5
27/06/2025	Finished Adder/Subtractor + Rewrote command explanations	4
06/07/2025	Update Adder/Subtractor + Create Comparator.dig + Update Commands	4
27/07/2025	Changed timing for a few files + Update Adder/Subtractor + Comparator	5
06/08/2025	Started work on the ALU + Finish Incrementor + Update Commands	5
07/08/2025	Update Incrementor.dig + Create Shift Register	6
08/08/2025	Update commands.xlsx + Create Flag Register.dig	5
15/08/2025	Update Flag Register.dig + Update commands.xlsx	4
20/08/2025	added STC and STF outputs to the adder/subtractor + Create Register.dig + Update Commands.xlsx	3
21/08/2025	Update Registers + Update Commands	6
02/09/2025	added program counter	8
04/09/2025	Update CPU.dig + Update commands.xlsx	4
01/10/2025	Update commands.xlsx	4
02/10/2025	Created the 8-to-16-bit converter	5
06/10/2025	quality of life + Update Shift Register.dig	3
07/10/2025	Add Barrel Shift Register + add RAM	3
09/10/2025	Update RAM.dig + named Commands + made Opcodes Hexadecimal	3
10/10/2025	Update Incrementor.dig + Update Adder Subtractor.dig	2
11/10/2025	2 new commands + small fixes + added Opcodes to Commands.xlsx	4
12/10/2025	Update Adder Subtractor.dig + removed some redundant commands	2
13/10/2025	adjusted command values + changes to incrementor + started adding lookup table for the flash	4
14/10/2025	changed the flash control logic, instead added data bus disable line	3
15/10/2025	changed RAM.dig	2

19/10/2025	added data to address bus interface + Update commands.xlsx	3
20/10/2025	added i/o	2
20/10/2025	changed LN line to a direct connection, as it is not a bus	2
24/10/2025	Update RAM.dig + added input control + adjusted timing for bus disabling + added shift distance reg + changed the input leds so that they show the value stored in the input register	9
28/10/2025	Update CPU.dig	3
09/11/2025	Update commands.xlsx + coded a short test programme + found 2 short circuits within the alu + fixed the ram and increased the downtime on the address and data busses + Update CPU.dig	9
13/11/2025	worked on the test programme	7
14/11/2025	checked flash	3
15/11/2025	Think through the timing of the Flag Register timing	7
16/11/2025	worked on the test programme	6
20/11/2025	Fixed errors + marked all the tested commands	5
05/12/2025	worked on the test programme	8
08/12/2025	worked on the test programme	4
08/12/2025	added every command to the test	5
08/12/2025	Fixed errors + Added 8 by 8 Pixel Display	5
09/12/2025	changes to commands	3
11/12/2025	added controller inputs + created Programme for the screen	4
08/01/2026	started writing the assembler	5
10/01/2026	changes test programme + changes mistakes layout.txt	3
22/01/2026	continued working on the Assembler	3
29/01/2026	work on the Assembler	6
31/01/2026	Testing the Assembler	3
05/02/2026	PCB Assembly preparation	3
06/02/2026	PCB Assembly Control Module	6
10/02/2026	PCB Assembly I/O Module	4
11/02/2026	PCB Assembly Memory Module	3
12/02/2026	PCB Assembly ALU + Work on Assembler	10
13/02/2026	PCB debugging + Work on Assembler + changed the flash in the simulation	6
14/02/2026	PCB debugging	4
15/02/2026	Documentation	5
24/02/2026	PCB debugging	3
25/02/2026	PCB debugging	6
26/02/2026	PCB debugging	3
27/02/2026	PCB debugging	5
03/03/2026	PCB debugging	4
05/03/2026	PCB debugging	5
06/03/2026	PCB debugging	5
24/02/2026	fix the test programme	5
27/02/2026	added visualization for the busses in the simulation + Documentation	6
28/02/2026	Documentation, CPU layout changes	6
01/03/2026	Documentation	7
06/03/2026	Documentation	7
09/03/2026	Documentation	9

10/03/2026	Documentation	6
11/03/2026	Documentation	8
12/03/2026	Documentation	8
13/03/2026	Documentation	4
14/03/2026	Documentation + added functionality to the assembler	5
15/03/2026	Documentation + new calculator programme + added functionality to the assembler + started implementing labels	12
16/03/2026	Work on labels + Extend delay in DAT to ADR interface.dig to allow it to actually be used in programmes + add label to code	6
17/03/2026	Documentation	4
18/03/2026	fixed code + fixed error output in main.cpp	3
19/03/2026	Documentation	6
20/03/2026	Documentation	5
21/03/2026	Documentation	7
22/03/2026	Documentation	6
24/03/2026	Documentation	5
25/03/2026	Documentation	6
	Sum:	**475**

Table 39: Time Table Florian Silberbauer

7.1.2 Simon Schmidtgrabner

Date	Task	Hours Spent
04/09/2025	parts	3
06/09/2025	Partslist, added XOR, AND, OR	5
08/09/2025	parts	4
09/09/2025	Partslist, little format changes	4
29/09/2025	parts	5
01/10/2025	Partslist,	4
02/10/2025	Partslist, Buffer, parts	5
03/10/2025	parts, Kicad, added Parts into Kicad	4
04/10/2025	parts, Kicad	5
05/10/2025	KicadParts, partlist	5
06/10/2025	Parts, added new NOR flash M29F800FT	3
07/10/2025	Kicad,parts	5
10/10/2025	partsList, parts,Kicad	4
11/10/2025	parts, Kicad	5
12/10/2025	partsList, Kicad, added Register 1 0x20 and 0x21, finished Incrementor	3
12/10/2025	Kicad, added Register 2(0x22 u 0x23) and 3(0x13 u 0x14) added BarrelShiftRegister added AdderSubstractor added cmd Comparators to the bitwise and binary Logic	2
14/10/2025	Kicad	3
15/10/2025	Programmer Xgecu T76, added the exe of the T76 software	2
17/10/2025	Kicad	4
18/10/2025	Kicad, added the CMD comparators for Inc, Comp, AddSub, FlagReg, BarrelShifter, ProgCount added Diodes and DATdis line	6
23/10/2025	Parts, added male/ female headers for PCB to PCB	4
24/10/2025	Kicad,parts, worked on Flash memory began RAM	4
25/10/2025	Kicad, finished flash mem added DATtoADR	6
27/10/2025	Kicad, parts, added Input Output changed Q bus to DAT bus	6
29/10/2025	Kicad, parts, added Step-Down Conv circuit for 20V VBUS to 5V VCC	7
01/11/2025	Kicad, changed to module version	5
06/11/2025	parts,Kicad, folder structure, added Clock Generator	3
09/11/2025	Kicad SCH/PCB, began with INOUTPUT module nearly done	5
27/11/2025	Kicad, parts	4
30/11/2025	Kicad,parts, created Flash mem and RAM PCB	5
07/12/2025	Kicad, added the Programme Counter und Dat to adr module	6
29/12/2026	Kicad ALU PCB	8
30/12/2026	Kicad ALU PCB	7
31/12/2026	Kicad ALU PCB	8
01/01/2026	Kicad ALU PCB	5
02/01/2026	Kicad ALU PCB	8
03/01/2026	Kicad ALU PCB	7
04/01/2026	Kicad ALU PCB	8
05/01/2026	Kicad ALU PCB	7
06/01/2026	Kicad ALU PCB	6
07/01/2026	Kicad, added Leds for B Bus and A Bus	3

08/01/2026	Kicad	3
08/01/2026	Kicad	4
10/01/2026	Kicad Fixes only schematic	4
10/01/2026	Kicad, PCB Fix, added GND panels on Top and Bot Layer	3
11/01/2026	Kicad separated PCBs, added production files	3
15/01/2026	sorted some things. added bom	2
22/01/2026	changed 150Ohms to 390	1
05/02/2026	PCB Assembly preparation	3
06/02/2026	PCB Assembly Control Module	6
10/02/2026	PCB Assembly I/O Module	4
11/02/2026	PCB Assembly Memory Module, updated T76 software, Kicad bugs	4
12/02/2026	PCB Assembly ALU, kicad incrementor cmd comp fix	9
13/02/2026	PCB debugging	4
14/02/2026	PCB debugging	4
24/02/2026	PCB debugging	3
25/02/2026	PCB debugging	6
26/02/2026	PCB debugging	3
27/02/2026	PCB debugging	5
03/03/2026	PCB debugging, Documentation	4
05/03/2026	Kicad, PCB debugging, Documentation	5
06/03/2026	PCB debugging, Documentation	5
07/03/2026	Documentation	2
08/03/2026	Documentation	2
09/03/2026	Documentation	1
10/03/2026	Documentation	3
11/03/2026	Documentation	3
13/03/2026	Documentation	2
15/03/2026	Documentation, Fixed some Design bugs	3
17/03/2026	Documentation, part 1 of changing to A4	5
18/03/2026	Documentation	3
19/03/2026	changed page numbers, compressed ROOT file from A1 to A4, Docu	3
20/03/2026	Documentation	6
21/03/2026	changed page numbers, every schematic is now A4	3
22/03/2026	Documentation	4
23/03/2026	Documentation	2
24/03/2026	Documentation	3
25/03/2026	Documentation	3
	Sum:	**336**

Table 40: Time Table Simon Schmidtgrabner

8 Indexes

8.1 List of References

Adafruit Industries, 2025. *HUSB238 USB Type-C Power Delivery Breakout (Product 5807).* [Online]
Available at: https://www.adafruit.com/product/5807
[Accessed 12 Mar 2026].

BenEater, 2016. *Youtube.* [Online]
Available at: https://www.youtube.com/watch?v=HyznrdDSSGM&list=PLowKtXNTBypGqImE405J2565dvjafglHU
[Accessed 2023].

Bourns, REV. 03/21. *CR Series - Thick Film Chip Resistors,* https://www.mouser.at/datasheet/3/40/1/cr.pdf: C1753 05/17/18R.

InductiveLoad, 2009. *Logic Gate.* [Online]
Available at: https://commons.wikimedia.org/wiki/File:AND_ANSI_Labelled.svg
[Accessed 15 Febuary 2026].

InductiveLoad, kein Datum *Wikipedia.* [Online]
Available at: https://commons.wikimedia.org/wiki/File:AND_ANSI_Labelled.svg

Monolithic Power, 12/11/2020. *MP2338 Rev. 1.0,* https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP2338GTL: Monolithic Power.

Texas Instruments, Feb. 2000 [Revised Jan. 2015]. *LM555 Timer datasheet (Rev. D),* https://www.ti.com/lit/ds/symlink/lm555.pdf?ts=1772877948918&ref_url=https%253A%252F%252Fwww.ecosia.org%252F: SNAS548D.

Wdwd, Stündle, 2011. *File:Pierce oscillator.svg(CC BY-SA 3.0).* [Online]
Available at: https://commons.wikimedia.org/w/index.php?title=File:Pierce_Quarz_Oszillator.svg&oldid=457037368
[Accessed 7 March 2026].

Wikipedia contributors, 2026. *7400-series integrated circuits.* [Online]
Available at: https://en.wikipedia.org/w/index.php?title=7400-series_integrated_circuits&oldid=1340170561
[Accessed 10 Mar 2026].

Wikipedia contributors, 2026. *List of 7400-series integrated circuits.* [Online] Available at: https://en.wikipedia.org/w/index.php?title=List_of_7400-series_integrated_circuits&oldid=1334497186
[Accessed 10 Mar 2026].

Wikipedia contributors, 3 March 2026. *E series of preferred numbers,* https://en.wikipedia.org/w/index.php?title=E_series_of_preferred_numbers&oldid=1341442161: Wikipedia, The Free Encyclopedia..

8.2 List of Images

8.3 List of Tables

8.4 List of Formulas

8.5 List of Abbreviations

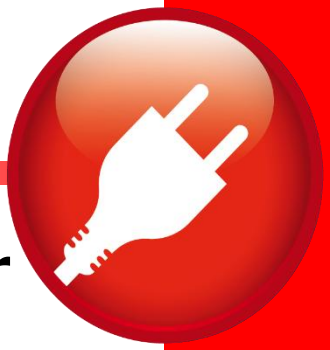
Bit	a single value which can either be 0 or 1 often used in Computer Science
Byte	a combination of 8 Bits
RAM	(random access storage) a type of volatile storage often used in computers
ALU	(arithmetic logic unit) a circuit within a computer which handles calculations
IC	(integrated circuit) Set of electronic circuits on a small flat surface
t_{pd}	(propagation delay) for signal to change state from in- to output of an IC
Stt	Schmitt trigger
Unb	Unbuffered
PD	Power Delivery

HÖHERE TECHNISCHE BUNDESLEHRANSTALT WELS

WWW.HTL-WELS.AT

DIPLOMA THESIS

8Bit-Computer



Higher Department for

Electrical Engineering

8Bit-Computer

htl bildung mit zukunft

2025/26